

C++

Programming:

An Object-Oriented Approach

Chapter 19

Standard Template Library (STL)

C++

Programming:

An Object-Oriented
Approach



BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

Objectives Part 1

To list the four components of the STL: containers, algorithms, iterators, and function objects.

To discuss iterators and explain their different types.

To discuss sequence containers (*vector*, *deque*, and *list*) that define collections of items organized as sequences.

To discuss container adapters (*stack*, *queue*, and *priority_queue*) as special cases of sequence containers in which items can be accessed, inserted, or erased only from the front or the back of the container.

Objectives Part 2

To discuss associate containers (*set* and *map*) as sorted elements in which an element can be accessed using a key.

To discuss functions and function objects and how to customize them for use in algorithms that we discuss last in the chapter.

To discuss selected generic algorithms defined in the STL such as non-mutating algorithms, mutating-algorithms, sorting and related algorithms, and numeric algorithms.

INTRODUCTION

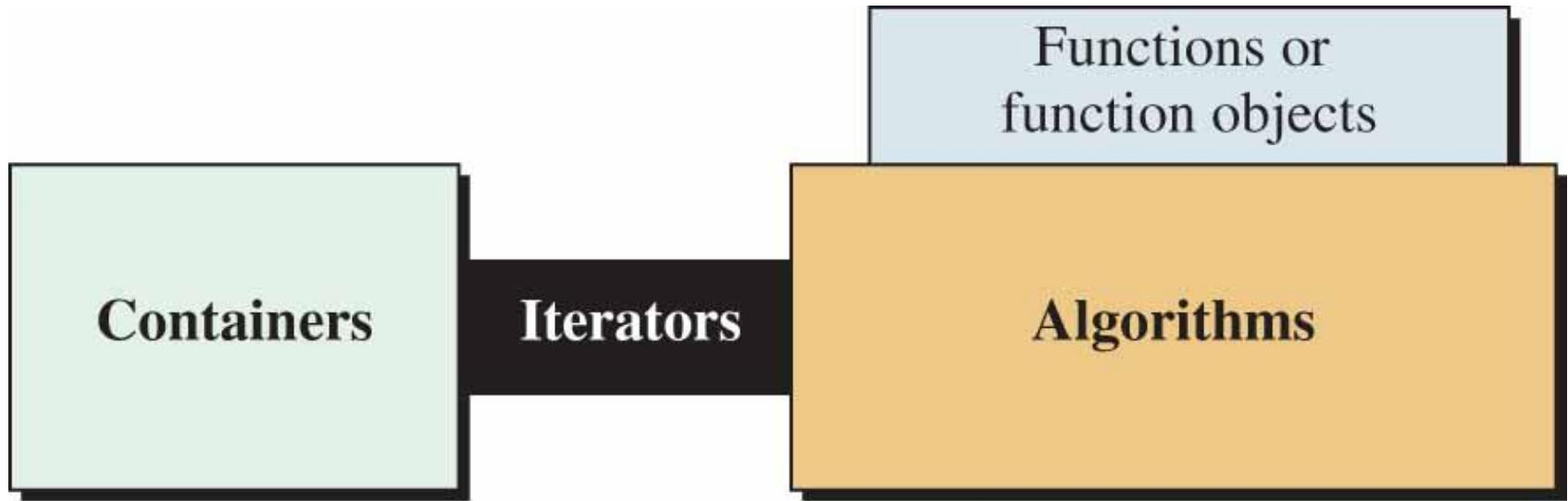
The Standard Template Library (STL) is the result of years of research to solve two important issues:

- 1. reusability of software.**
- 2. separation of functionality.**

Components Part 1

STL is made of four components as shown in Figure 19.1.

Figure 19.1 *STL Components*



Four components of STL

Components Part 2

Containers

Containers are the heart of the STL.

The library defines four categories of containers, each for a different purpose:

sequence containers.

associate containers.

container adapters.

pseudo-containers.

We discuss only the first three categories in this chapter.

In the fourth category, pseudo-containers, we discuss only the *bitset* class.

Components Part 3 ₍₁₎

Algorithms

Algorithms are operations that we need to apply to the container elements.

They are divided into four categories:

non-mutating algorithms that do not change the container structure.

mutating algorithms that change the structure

sorting algorithms that reorder elements in a container

numeric algorithms that apply mathematical operations on the elements.

Components Part 3 ₍₂₎

Iterators

Research and experience have shown that the operations we need to perform on the elements of a container are independent from the type of the element and the type of the container.

We need to access the elements in the container one by one.

The iterators allow us to access each element individually and apply the desired operation to it.

This means that we do not need to have one algorithm that counts one type and another that counts another type.

Components Part 4

Functions and Function Objects.

To apply algorithms on container, STL uses functions or function objects in the definition of the algorithms.

They allow the STL to define a generic algorithm and use function or function object to make it specific.

In the first case, we need a function definition; in the second case, we need a class for which the *operator()* is defined.

The user defines a function for the first case, but the class is normally defined in the STL library and the user can only call the constructor of the class.

ITERATORS

We learned that we can declare a *pointer*, a system-defined type with a number of operations, that points to an element of a container such as an array or a linked list.

We can then use the operations defined for pointers to access the elements of the container one by one.

Introduction Part 1

An *iterator* is an abstraction of a pointer.

It is a class type that has a pointer as a data member with predefined operations that can be applied to the pointer member.

We can instantiate an object of an iterator and then apply the operations defined for it.

Introduction Part 2

One benefit of an iterator over a pointer is that we can not limit the operations defined for a pointer nor can we augment the operations defined for a pointer.

But we can do both for an iterator.

For example, we can define an iterator that can move only forward by including the ++ operator and excluding the -- operator.

We can define an iterator that cannot jump from one object in a container to another by not defining the plus (+) or the minus (–) operator for the class.

We can say that an iterator can only access an element or can only change an element, or can do both.

Introduction Part 3

Another advantage of an iterator is that it can hide the internal structure of a container.

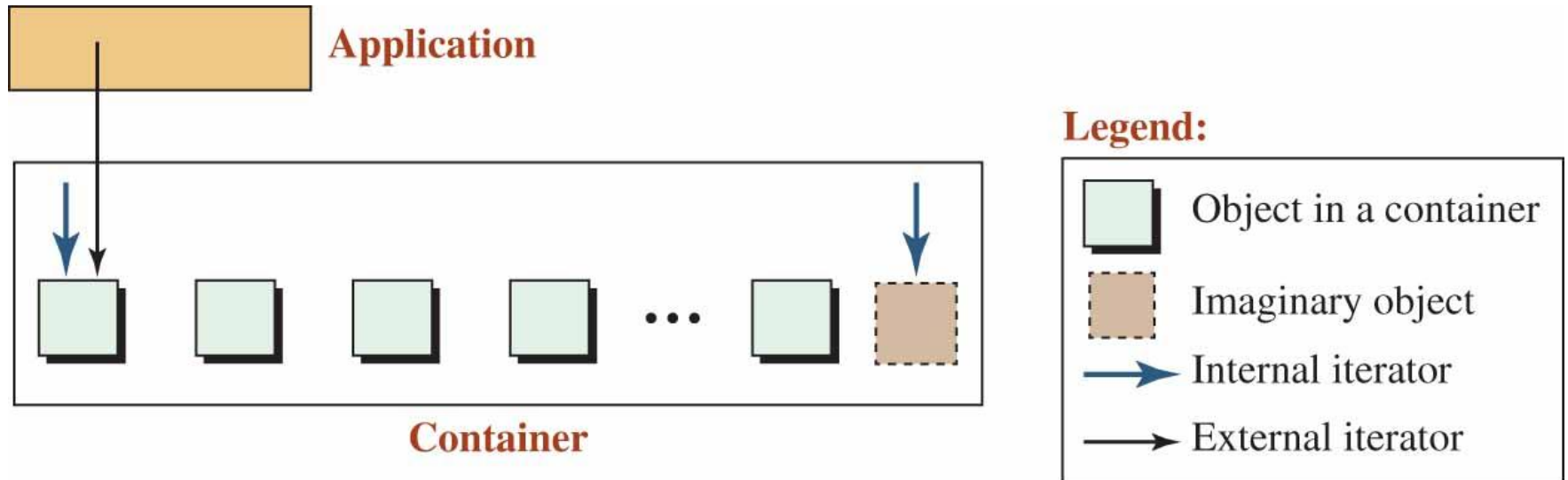
Each container can define its own iterator type whose design is hidden from the user but the user can create an iterator of that type and access the objects in the container.

The way iterators work with most containers is that the STL defines several internal iterators that are fixed and cannot move.

They are used to control the external iterators instantiated by the applications. Figure 19.2 shows an example.

Introduction Part 4₍₁₎

Figure 19.2 *Internal and external iterator*



Introduction Part 4 ₍₂₎

The left internal iterator is pointing to the first object in the container.

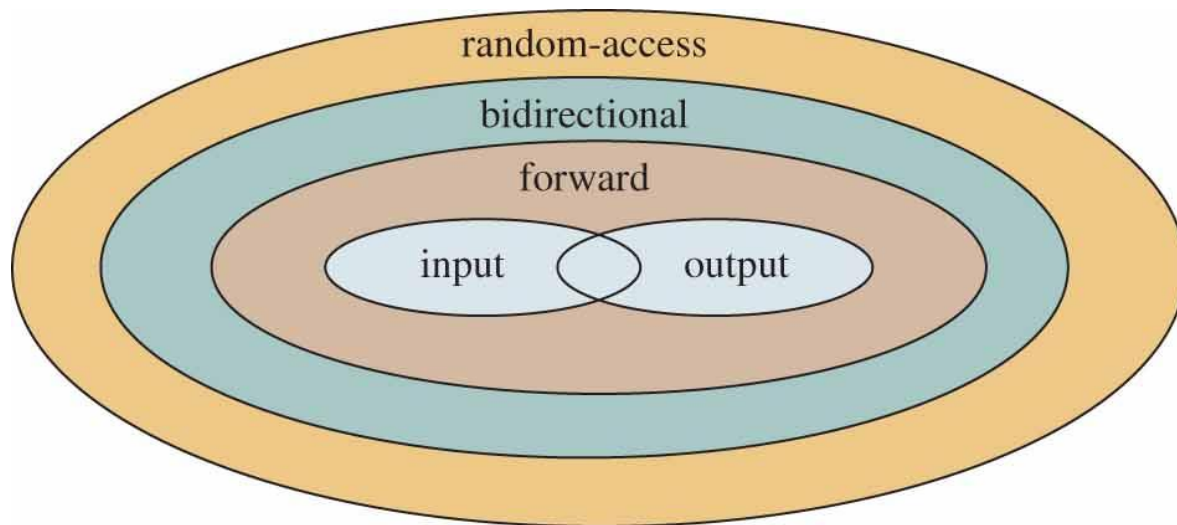
The external iterator is initialized to point where the left internal iterator is pointing.

The application can move the external iterator through the container until it reaches the right internal iterator, which is pointing to a non-existent object at the end.

Five Types of Iterators: Part 1

We can categorize iterators into five types: *input iterator*, *output iterator*, *forward iterator*, *bidirectional iterator*, and *random-access iterator* as shown in Figure 19.3.

Figure 19.3 *Hierarchy of iterator types*



Abbreviations

InIter input

OutIter output

FwdIter forward

BidIter bidirectional

RandIter random-access

Five Types of Iterators: Part 2

Table 19.1 shows five iterator types and their supported operations.

Table 19.1 *Supported operations for iterators*

Iterator	read	Write	*	++	--	== !=	<, <=, >, >=	+ -
input	√		√	√		√		
output		√	√	√		√		
forward	√	√	√	√		√		
bidirectional	√	√	√	√	√	√		
random-access	√	√	√	√	√	√	√	√

Five Types of Iterators Part 3 ₍₁₎

Input Iterator.

An *input iterator* can use the dereference operator only to read from a container; it is not allowed to write to it. In other words, an *input iterator* treats the container as a source of data items to read.

Output Iterator.

An *output iterator* can use the dereference operator to only write to a container; it is not allowed to read from it.

Five Types of Iterators Part 3 ₍₂₎

Forward Iterator.

A forward iterator can read or write elements. Its functionality is the combination of the input and output iterator.

Bidirectional Iterator.

A bidirectional iterator can move in both directions: backward and forward. The ++ and -- operators are defined for this iterator.

Five Types of Iterators Part 3 ₍₃₎

Random-Access Iterator.

A random-access iterator has the capabilities of a bidirectional iterator, and in addition it supports the add (+) operator and the subtract operator (–).

It also provides four relational operators (<, <=, >, and >=) that are not provided by the other iterators.

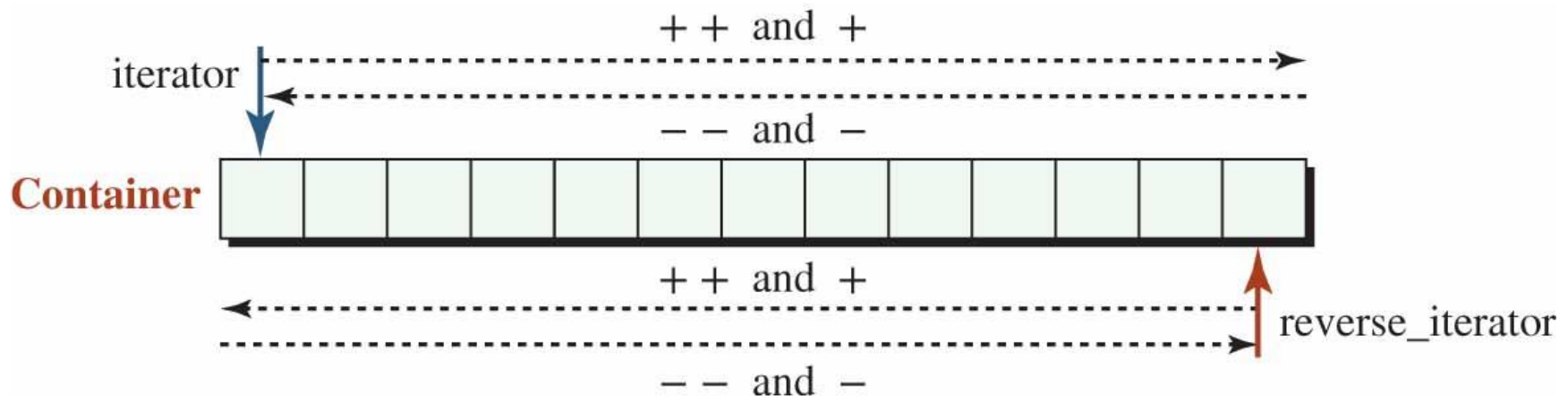
These operators allows us to use the index operator [] which requires the +, – and relational operators which are needed to make the iterator jump forward or backward.

Moving Direction

A container normally defines two categories of iterators: regular (called *iterator*) and reverse (called *reverse_iterator*). The moving direction for these two types of iterators need to be understood.

It is shown in Figure 19.4.

Figure 19.4 *Moving directions of a regular and a reverse iterator*



Constantness Part 1

A container can also define two types of iterators: *const iterator* and *const_iterator*.

Type: const iterator.

The type *const iterator* defines an iterator type which is a constant object.

In other words, it cannot be changed after it is created.

We cannot move the *const iterator* to point to another element.

It is like the name of an array which is a constant pointer.

Constantness Part 2

Type: `const_iterator`.

The type *`const_iterator`* defines an iterator type in which the dereferenced item is an *rvalue* (cannot be changed by the iterator).

This is similar to when we declare an array whose elements are constants.

SEQUENCE CONTAINERS

A sequence container is a collection of objects in which the programmer controls the order of storing and retrieving elements.

The STL provides three sequence containers: *vector*, *deque*, and *list*.

The first two are normally implemented as dynamic arrays; the third is normally implemented as a doubly linked list.

Public Interface Part 1 ₍₁₎

Table 19.2 is a general public interface of sequence containers.

The abbreviation SC must be replaced with one of the three sequence containers when the corresponding column is ticked.

Each of these containers is a template class in which T defines the type of the element in the container.

The *typename* T can be any built-in or user-defined type.

We have used *iter* for an iterator, *inlter* for an input iterator, and *pred* for a Boolean function returning *true* or *false*.

Public Interface Part 2 ₍₁₎

The abbreviation V stands for *vector*, D for *deque*, and L for *list*.

Table 19.2 *Public Interface of sequence containers*

Constructors, assignment, and destructor	V	D	L
SC <T> :: SC ()	✓	✓	✓
SC <T> :: SC (size_type <i>n</i> , const T& <i>value</i> = T())	✓	✓	✓
SC <T> :: SC (const_iter <i>first</i> , const_iter <i>last</i>)	✓	✓	✓
SC <T> :: SC (const SC <T>& <i>other</i>)	✓	✓	✓
SC <T>& SC <T> :: operator =(const SC <T> & <i>other</i>)	✓	✓	✓
SC <T> :: ~ SC ()	✓	✓	✓

Public Interface Part 3 ₍₁₎

Table 19.2 *Public Interface of sequence containers*

Size and capacity	V	D	L
<code>size_type SC <T> :: size()</code>	✓	✓	✓
<code>size_type SC <T> :: max_size()</code>	✓	✓	✓
<code>void SC <T> :: resize(size_type n, T value = T())</code>	✓	✓	✓
<code>bool SC <T> :: empty()</code>	✓	✓	✓
<code>size_type SC <T> :: capacity()</code>	✓		
<code>void SC <T> :: reserve(size_type, n)</code>	✓		
Accessing elements (constant and non constant versions)	V	D	L
<code>T& SC <T> :: front()</code>	✓	✓	✓
<code>T& SC <T> :: back()</code>	✓	✓	✓
<code>T& SC <T> :: operator[](size_type index)</code>	✓	✓	
<code>T& SC <T> :: at(size_type index)</code>	✓	✓	
Iterators (regular and constant_iterator versions)	V	D	L
<code>iter SC <T> :: begin()</code>	✓	✓	✓
<code>iter SC <T> :: end()</code>	✓	✓	✓
<code>reverse_iter SC <T> :: rbegin()</code>	✓	✓	✓
<code>reverse_iter SC <T> :: rend()</code>	✓	✓	✓

Public Interface Part 4 ₍₁₎

Table 19.2 *Public Interface of sequence containers*

Insertion	V	D	L
<code>void SC <T> :: push_front(const T& value)</code>		✓	✓
<code>void SC <T> :: push_back(const T& value)</code>	✓	✓	✓
<code>iter SC <T> :: insert(iter pos, const T& value)</code>	✓	✓	✓
<code>void SC <T> :: insert(iter pos, size_type n, const T& value)</code>	✓	✓	✓
<code>void SC <T> :: insert(iter pos, InIter first, InIter last)</code>	✓	✓	✓
Erasure	V	D	L
<code>void SC <T> :: pop_front()</code>		✓	✓
<code>void SC <T> :: pop_back()</code>	✓	✓	✓
<code>iter SC <T> :: erase(iter pos)</code>	✓	✓	✓
<code>iter SC <T> :: erase(iter first, iter second)</code>	✓	✓	✓
<code>void SC <T> :: clear()</code>	✓	✓	✓
<code>void SC <T> :: remove(const T& value)</code>			✓
<code>void SC <T> :: remove_if(pred p)</code>			✓
<code>void SC <T> :: unique(pred p)</code>			✓

Public Interface Part 5 ₍₁₎

Table 19.2 *Public Interface of sequence containers*

Splice, merge, and sort	V	D	L
<code>void SC <T> :: splice(iter pos, SC<T> other)</code>			✓
<code>void SC <T> :: splice(iter pos, SC<T> other, iter other)</code>			✓
<code>void SC <T> :: splice(iter pos, SC<T> other, iter i, iter j)</code>			✓
<code>void SC <T> :: merge(SC<T> other)</code>			✓
<code>void SC <T> :: sort()</code>			✓
Swapping	V	D	L
<code>void SC <T> :: swap(SC<T>& other)</code>	✓	✓	✓
Global Functions (op means <, <=, >, >=, == or !=)	V	D	L
<code>bool operator op(const SC<T> left, const SC<T> right)</code>	✓	✓	✓
<code>void swap(SC<T>& left, SC<T>& right)</code>	✓	✓	✓

Note that each accessing member also has a constant version that creates an *rvalue*, but we did not include them to make the interface eshorter.

Also note that ach iterator has a *const_iterator* version that we did not include for the same reason.

The Vector Class Part 1

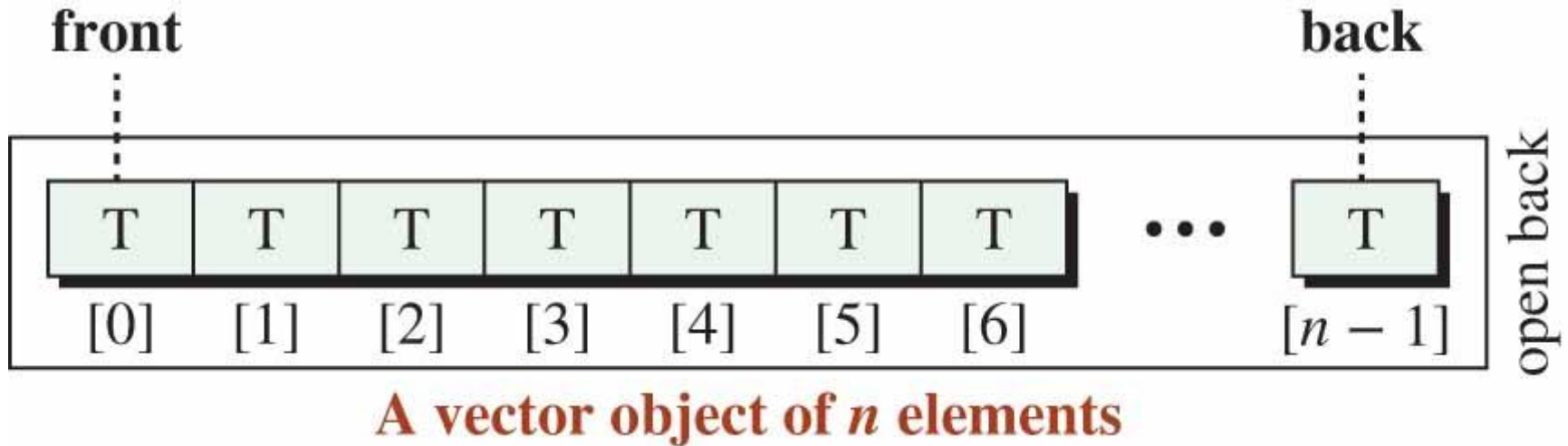
The vector `<T>` class, defined in the `<vector>` header, implements a *sequence container* that provides fast random access to any element and fast insertion and deletion at the back.

This means that if we need a sequence container with a lot of insertions and deletions at the front or the middle, a vector is not the best choice.

A vector is a sequence of elements with an open end as shown in Figure 19.5.

The Vector Class Part 2

Figure 19.5 *A vector as a sequence structure with indexes*



The Vector Class Part 3

Operations

The syntax of member functions for the *vector* class can be obtained from the general public interface (Table 19.2).

Constructors and Assignment Operator.

The vector class has one default constructor, two parameter constructors, one copy constructor, and one assignment operator.

The following shows how we use these member functions.

The Vector Class Part 4

```
vector <T> vec;                // Constructs an empty vector
vector <T> vec(4 , value);      // Constructs a vector of 4 elements
vector <T> vec(from, to);       // A vector created from
vector <T> vec(otherVec);       // Copy constructor
vector <T> vec = otherVec;      // Assignment operator
```

The first line shows how to create an empty vector.

The second line shows how to create a vector of four elements all of the same value.

The third line shows how to copy elements from another structure to make a vector.

The fourth line is a copy constructor. The last line is the assignment operator.

The Vector Class Part 5

Destructor

The destructor is called automatically when a vector container goes out of scope.

The destructor is designed to delete the whole array from the heap.

Size and Capacity.

The vector class has six member functions related to the size and capacity of a vector.

<code>vec.size();</code>	<code>// Returns the current size</code>
<code>vec.max_size();</code>	<code>// Returns the maximum size</code>
<code>vec.resize(n, value);</code>	<code>// Resizes the vector</code>
<code>vec.empty();</code>	<code>// Returns true if vector is empty</code>
<code>vec.capacity();</code>	<code>// Returns potential size</code>
<code>vec.reserve(n);</code>	<code>// Reserves more memory locations</code>

The Vector Class Part 6

Accessing Elements (for retrieve or change).

The vector class provides the following member functions to access elements already in the vector.

```
vec.front() ;           // Access the first element
vec.back() ;            // Access the last element
vec [i] ;               // Access the element at index i
vec.at(i) ;             // Access the element at index i
```

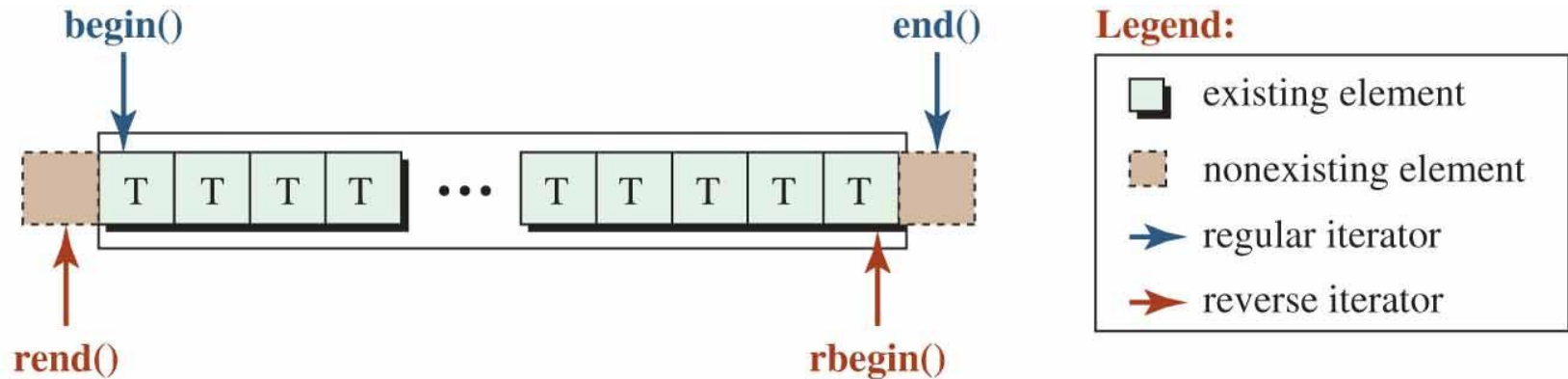
Iterators

The vector class defines two regular iterators and two reverse iterators although we have shown only the regular ones in Table 19.2.

In each category, we can have a constant and a non-constant operator. Figure 19.6 shows these iterators.

The Vector Class Part 7

Figure 19.6 *Iterators and reverse iterators in a vector*



Notes:

The **begin()** function returns a regular iterator that points to the front element.

The **end()** function returns a regular iterator to a nonexistent element after the back element.

The **rbegin()** function returns a reverse iterator that points to the back element.

The **rend()** function returns a reverse iterator to a nonexistent element before the front element.

Both iterators are random-access iterators that can jump forward and backward.

The Vector Class Part 8 ₍₁₎

To use iterators in our program, we need first to instantiate them.

Since each container has its own iterators, we need to create one or both of the iterators for that specific class as needed.

The Vector Class Part 8 ₍₂₎

```
vector <T> :: iterator iter;           // A regular iterator  
vector <T> :: reverse_iterator riter;  // A reverse iterator
```

After instantiation of the user iterators, we need to set them.

The iterator *iter* needs to start where the iterator returned from *begin()* is set; we need to set the iterator *riter* to where the iterator returned from *rbegin()* is set as shown below:

```
iter = vec.begin();  // iter starts when the vec.begin() points to  
riter = vec.rbegin();  // riter starts when the vec.rbegin() points to
```

We create a vector of size 10 and use regular and reverse iterators to access elements (Program 19.1).

Public Interface Part 6 ₍₁₎

Note that each accessing member also has a constant version that creates an *rvalue*, but we did not include them to make the interface shorter.

Also note that each iterator has a *const_iterator* version that we did not include for the same reason.

Testing Iterators in a Vector Part 1

Program 19.1 Testing iterators in a vector

```
1  /*****
2  * A simple program to test regular and reverse navigation      *
3  *****/
4  #include <vector>
5  #include <iostream>
6  #include <iomanip>
7  using namespace std;
8
9  int main()
10 {
11     // Constructing a vector of 10 elements and two iterators
12     vector<int> vec(10);
13     vector<int> :: iterator iter;
14     vector<int> :: reverse_iterator rIter;
15     // Changing the value of elements
16     for (int i = 0; i < 10; i++)
17     {
18         vec.at(i) = i * i;
19     }
20     // Printing the elements using the forward iterator
```


Testing Iterators in a Vector Part 2

Program 19.1 Testing iterators in a vector

```
21     cout << "Regular navigation: ";
22     for (iter = vec.begin() ; iter != vec.end() ; ++iter)
23     {
24         cout << setw(4) << *iter;
25     }
26     cout << endl;
27     // Printing the elements using reverse iterator
28     cout << "Reverse navigation: ";
29     for (rIter = vec.rbegin() ; rIter != vec.rend() ; ++rIter)
30     {
31         cout << setw(4) << *rIter;
32     }
33     cout << endl;
34     return 0;
35 }
```

Run:

```
Regular navigation:  0   1   4   9  16  25  36  49  64  81
Reverse navigation: 81  64  49  36  25  16   9   4   1   0
```

Testing Iterators in a Vector Part 3 ₍₁₎

We are using ++ operators for both the regular iterator and the reverse iterator.

In the first case, the iterator is moving from the front to the back of the vector; in the second case, it is moving from the back to the front of the vector.

At line 22, the regular iterator starts at the element returned from *vec.begin()* and stops before the element returned from *vec.end()*.

Testing Iterators in a Vector Part 3 ₍₂₎

At line 29, the reverse iterator does the same but uses the iterators returned from *vec.rbegin()* and *vec.rend()*.

Note that we are using ++ in both cases. Both iterators are moving forward (from beginning to the end or from end to the beginning).

To show that the iterators for a vector are in fact random access iterators (they can jump in both direction), we create a vector with the following elements:

0 10 20 30 40 50 60 70 80 90

Showing Random-access Nature of Iterators Part 1

Program 19.2 Showing the random-access nature of iterators

```
1  /*****
2   * A simple program to test random-access iterators in both      *
3   * regular and reverse iterators                                *
4   *****/
5  #include <iostream>
6  #include <vector>
7  using namespace std;
8
9  int main()
10 {
11  // Instantiation of a vector and two iterators
12      vector <int> vec;
13      vector <int> :: iterator iter1;
14      vector <int> :: reverse_iterator iter2;
15      // Filling the vector with 10 elements
16      for (int i = 0; i < 10; i++)
17      {
18          vec.push_back(i * 10);
19      }
20  // Using the regular iterator to print 40 followed by 20
```

Showing Random-access Nature of Iterators Part 2

Program 19.2 Showing the random-access nature of iterators

```
21     cout << "Printing 40 followed by 20" << endl;
22     iter1 = vec.begin();
23     iter1 += 4;
24     cout << *iter1 << " ";
25     iter1 -= 2;
26     cout << *iter1 << endl;
27     // Using the reverse iterator to print 50 followed by 70
28     cout << "Printing 50 followed by 70" << endl;
29     iter2 = vec.rbegin();
30     iter2 += 4;
31     cout << *iter2 << " ";
32     iter2 -= 2;
33     cout << *iter2 << endl;
34     return 0;
35 }
```

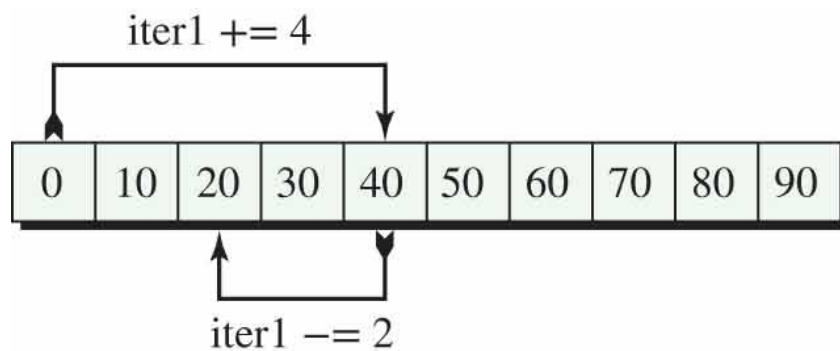
Run:

```
Printing 40 followed by 20
40 20
Printing 50 followed by 70
50 70
```

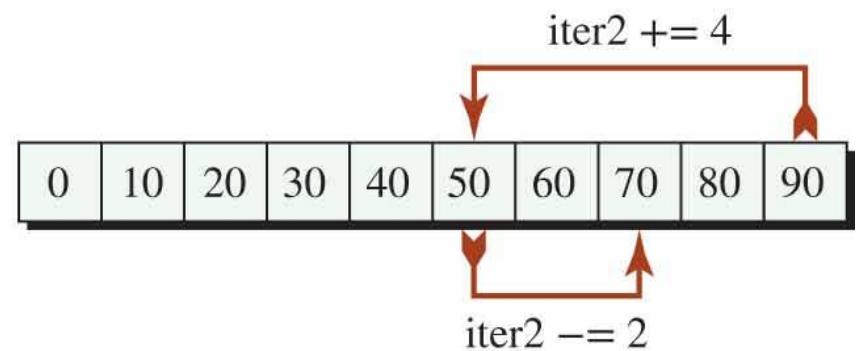
The Vector Class Part 9

Figure 19.7 shows how the regular and reverse iterators can move backward and forward randomly.

Figure 19.7 *Movement iterators in a vector*



Using regular iterator



Using reverse iterator

The Vector Class Part 10

Insertion

The vector defines several member functions to insert one or more items into the container.

The insertion at the back is very efficient and does not need relocation of the items in the vector.

Insertion in the middle and at the front require reallocation of items in memory.

The following show how we can use member functions to insert new elements into a vector.

Note that the last example copies the elements from the first to one before the last, but not the last element.

The Vector Class Part 11

```
vec.push_back(value);           // Insert value at the back
vec.insert(pos, value)         // Insert value before pos
vec.insert(pos, n, value);     // insert n copies before pos
vec.insert(pos, first, last);  // Insert another sequence
```

The *pos* parameter is the position pointed to by the defined iterator.

The *first* and *last* parameters are input iterators defining the range [*first*, *last*).

They can be used to copy the elements in the host vector or another container.

The Vector Class Part 12

Erasure

The vector defines several member function for erasure of one or more items in the container.

Erasure at the back is very efficient and does not need relocation of any items.

An erasure from the middle or from the front requires reallocation of memory and should be avoided.

The following shows the member functions we can use to erase elements from a vector.

```
vec.pop_back() ;           // Erase the back(last) element
vec.erase(pos) ;          // Erase the element before pos
vec.erase(first, second) ; // Erase elements in the range
vec.clear() ;              // Erase all elements
```

The Vector Class Part 13 (1)

Application

The vector class is designed to look like an array.

It has the same functionality as an array, but with some advantages.

First, the vector is a class with built-in type operations as we discussed.

Second, it is allocated in the heap and can be resized as needed.

The Vector Class Part 13 ₍₂₎

Application (continued)

Third, it has a very well-defined iterator mechanism that allows us to access, insert, and erase elements.

In summary, we can use a vector anytime we need an array and benefit from its advantages.

We can create a two-dimensional vector, which is a vector of vectors.

A vector of vectors has the same advantages over a two-dimensional array that a vector has over a one dimensional array.

The Vector Class Part 14

```
vector <type> table(rows, value);  
vector <vector <int> > table(rows, value);  
vector <vector <int> > table(rows, vector <int>(cols));
```

In the first line, we define a vector named *table* of *rows* elements, each initialized to *value*.

In the second line, we see that the type of the table is in fact a vector of integers.

In the third line, we see the value of each row is actually a vector of type integer and we have *cols* number of them.

Note that a space between the two symbols (> >) is needed to show that the combination is not the >> operator.

A Vector of Vectors (table) Part 1

Program 19.3 A vector of vectors(table)

```
1  /*****
2   * A program to simulate a table using a vector of vectors      *
3   *****/
4  #include <vector>
5  #include <iostream>
6  #include <iomanip>
7  using namespace std;
8
9  int main()
10 {
11     // Creation of a vector of vectors
12     int rows = 10;
13     int cols = 10;
14     vector < vector <int> > table(rows, vector <int>(cols));
15     // Changing values from default
16     for (int i = 0; i < rows ; i++)
17     {
18         for (int j = 0 ; j < cols; j++)
19         {
20             table [i][j] = (i + 1) * (j + 1);
```

A Vector of Vectors (table) Part 2

Program 19.3 A vector of vectors(table)

```
21     }
22 }
23 // Retrieving and printing values
24 for (int i = 0; i < rows ; i++)
25 {
26     for (int j = 0 ; j < cols; j++)
27     {
28         cout << setw(4) << table [i][j] << " ";
29     }
30     cout << endl;
31 }
32 return 0;
33 }
```

A Vector of Vectors (table) Part 3

Program 19.3 A vector of vectors (table)

Run:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

Ragged Vector To Create Pascal Triangle

Part 1

Program 19.4 Using a ragged vector to create a Pascal triangle

```
1  /*****
2  * A program to print Pascal coefficients *
3  *****/
4  #include <vector>
5  #include <iostream>
6  #include <iomanip>
7  using namespace std;
8
9  int main()
10 {
11     // Declaration
12     int power = 5;
13     vector < vector <int> > pascal(power + 1, vector <int>());
14     // Creation of ragged vector
15     for (int i = 0; i <= power; i++)
16     {
17         for (int j = 0; j < i + 1; j++)
18         {
19             pascal[i].push_back(0);
20         }
```


Ragged Vector To Create Pascal Triangle

Part 2

Program 19.4 Using a ragged vector to create a Pascal triangle

```
21     }
22     // Filling ragged table to create a Pascal triangle
23     for (int i = 0; i <= power ; i++)
24     {
25         for (int j = 0 ; j < i + 1; j++)
26         {
27             if (j == 0 || i == j)
28             {
29                 pascal [i][j] = 1;
30             }
31             else
32             {
33                 pascal [i][j] = pascal [i-1] [j-1] + pascal [i-1][j];
34             }
35         }
36     }
37     // Printing the triangle
38     for (int i = 0; i <= power ; i++)
39     {
40         cout << "Coefficients of (x + y)^( " << i << " ==> ";
```

Ragged Vector To Create Pascal Triangle

Part 3

Program 19.4 Using a ragged vector to create a Pascal triangle

```
41     for (int j = 0 ; j < i + 1; j++)
42     {
43         cout << setw(4) << pascal [i][j] << " ";
44     }
45     cout << endl;
46 }
47 return 0;
48 }
```

Run:

```
Coefficients of (x + y)^0 ==> 1
Coefficients of (x + y)^1 ==> 1  1
Coefficients of (x + y)^2 ==> 1  2  1
Coefficients of (x + y)^3 ==> 1  3  3  1
Coefficients of (x + y)^4 ==> 1  4  6  4  1
Coefficients of (x + y)^5 ==> 1  5 10 10 5  1
```

Ragged Vector To Create Pascal Triangle

Part 4

We define the number of rows (power + 1), but the size of the columns is set to zero.

The column in each row grows using the *push_back()* operation.

The values are set to 0, but they change in the inner *for* loop (lines 25 to 35).

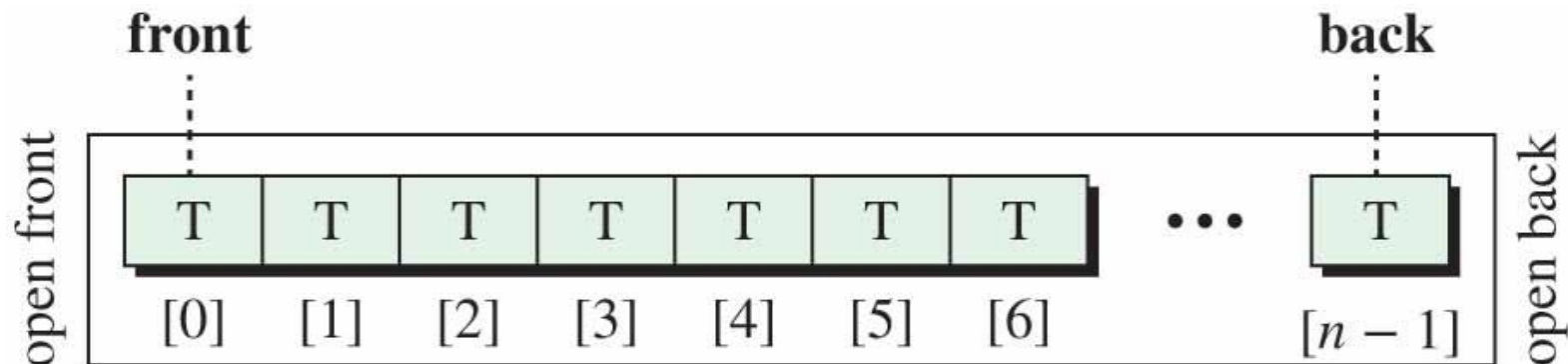
The deque Class Part 1

The *deque* $\langle T \rangle$ class, defined in the `<deque>` header file, is a sequence container similar to a *vector* but with two open ends.

The name *deque* is short for *double-ended queue*.

This means that we can insert and erase at both ends of a *deque* (Figure 19.9).

Figure 19.8 *A deque as a sequence structure with indexes*



A deque object of n elements

The deque Class Part 2

The operations for inserting and erasing elements from both ends are fast.

However, the extra capability to insert at the beginning makes a *deque* less efficient than a *vector* because it requires that the system allocate extra memory to be able to extend the *deque* at either end.

The deque Class Part 3 ₍₁₎

Operations

The syntax of member functions for the *deque* class can be obtained from the general public interface (Table 19.2).

They are very similar to the ones for the *vector* class with some minor differences.

We list only the differences.

The deque Class Part 3 ₍₂₎

Operations added for deque.

The *deque* class uses the same set of operations as the *vector*, but uses two additional operations.

```
deque.push_front(value);           // Insert value at the front  
deque.pop_front();                 // Delete value at the front
```

Operations missing in deque.

The *capacity()* and *reserve()* member function are missing in the *deque* class.

The reason is the implementation.

Since a *deque* can grow or shrink from both ends, the standard normally implements the *deque* as a set of blocks in heap memory.

The deque Class Part 4

Application

The *deque* class was designed to be the foundation for the *queue adapter* class that we discuss later.

However, it can be used in any application that needs insertion and deletion at both ends.

One application which normally uses a *deque* is *rotation*, in which we have a list of data items we want to rotate n times.

Clockwise rotation requires that we remove an element from the back and insert it at the front; counter-clockwise rotation requires that we remove an element from the front and insert it at the back.

Program 19.5 shows how we can rotate a list of names to the right and then to the left to get the original list.

Using deque to Rotate a List to Left and Right Part 1

Program 19.5 Using deque to rotate a list to the left and right

```
1  /*****
2  * A program to demonstrate rotation *
3  *****/
4  #include <deque>
5  #include <string>
6  #include <iostream>
7  #include <iomanip>
8  using namespace std;
9
10 // Global print function
11 void print(deque <string> deq)
12 {
13     for (int i = 0; i < deq.size (); i++)
14     {
15         cout << deq.at(i) << " ";
16     }
17     cout << endl;
18 }
19
20 int main()
```

Using deque to Rotate a List to Left and Right Part 2

Program 19.5 Using deque to rotate a list to the left and right

```
21 {
22     // Create a deque of five string and print it
23     deque<string> deq(5);
24     string arr [5]= {"John", "Mary", "Rich", "Mark", "Tara"};
25     for (int i = 0 ; i < 5; i++)
26     {
27         deq [i] = arr [i];
28     }
29     print(deq);
30     // Rotate the deque clockwise one element
31     deq.push_back(deq.front());
32     deq.pop_front();
33     print(deq);
34     // Rotate the deque counter-clockwise one element
35     deq.push_front(deq.back());
36     deq.pop_back();
37     print(deq);
38     return 0;
39 }
```

Using deque to Rotate a List to Left and Right Part 3

Program 19.5 *Using deque to rotate a list to the left and right*

Run:

John	Mary	Rich	Mark	Tara
Mary	Rich	Mark	Tara	John
John	Mary	Rich	Mark	Tara

The list Class Part 1

The *list* <T> class is a sequence container with fast insertion and deletion at any point.

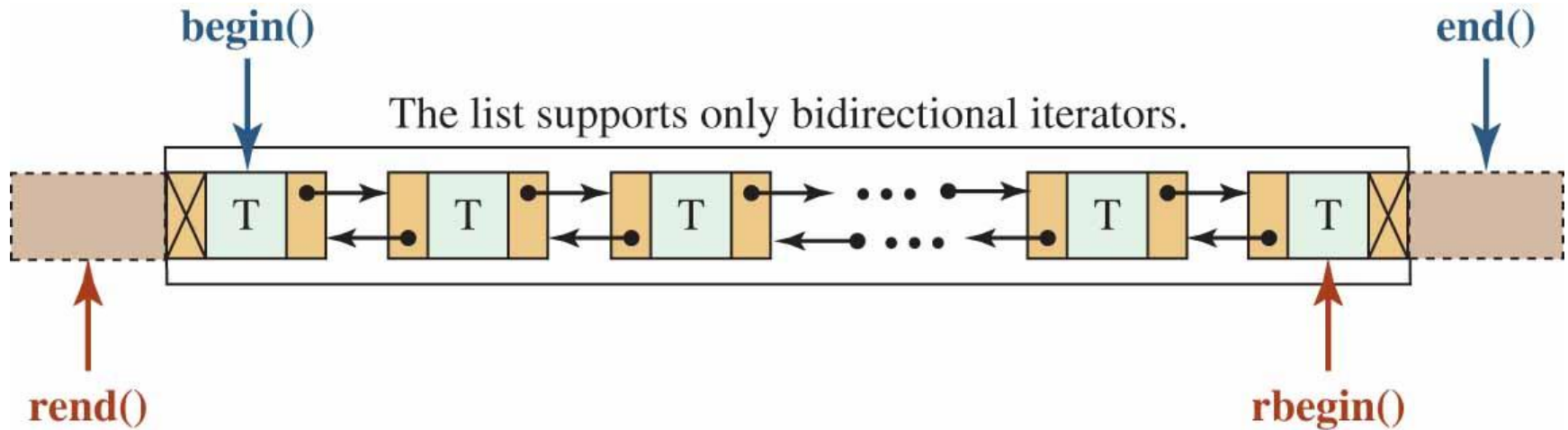
A *list* does not support random access for retrieving or changing the value of an element using the index operator or the *at()* member function because the *list* is implemented as a doubly linked list.

If we want to randomly access the list, we need to use an iterator that moves to the desired element and then accesses it.

Figure 19.9 shows a list made of five elements.

The list Class Part 2

Figure 19.9 *A list of elements*



Legend:

	existing element		regular iterator
	nonexisting element		reverse iterator
			pointer

The list Class Part 3 ₍₁₎

Operations

The syntax of member functions for the *list* class can be obtained from the general public interface (Table 19.2).

They are similar to the ones for the *vector* class and *deque* class with some differences.

We discuss only the differences.

Iterators

The *list* class supports only bidirectional iterators (not random-access).

This means that the *list* elements cannot be accessed using the index operator or the *at()* member function that requires the operators *+* and *-* defined for random access iterators.

The list Class Part 3 ₍₂₎

Capacity and Reserve.

The *capacity()* and *reserve()* member functions do not exist in the list class because the system adds an element at the front, at the back, and in the middle using the pointers in the double linked list.

Accessing Elements.

The operator [] and the *at()* function are not supported for the list class because the list class does not support the random-access operator (only bidirectional).

We need to explicitly use an iterator to access elements.

The list Class Part 4

Erasure

The list class provide three new operations to erase an element or elements from the list as the public interface shows.

The following shows how we call these operations.

```
remove(value) ;           // Erases all occurrences of value  
remove_if(predicate) ;    // Erases all occurrences if parameter is true  
unique(predicate) ;       // Erases duplicates if parameter is true
```


The list Class Part 5

Splice, Merge, and Sort.

The list class defines three other operations: *splice()*, *merge()*, and *sort()* as defined in the public interface.

```
splice(pos, first, last);           // Splice [first, last)
merge(other);                       // Merges two sorted lists
sort();                             // Sorts the list.
```

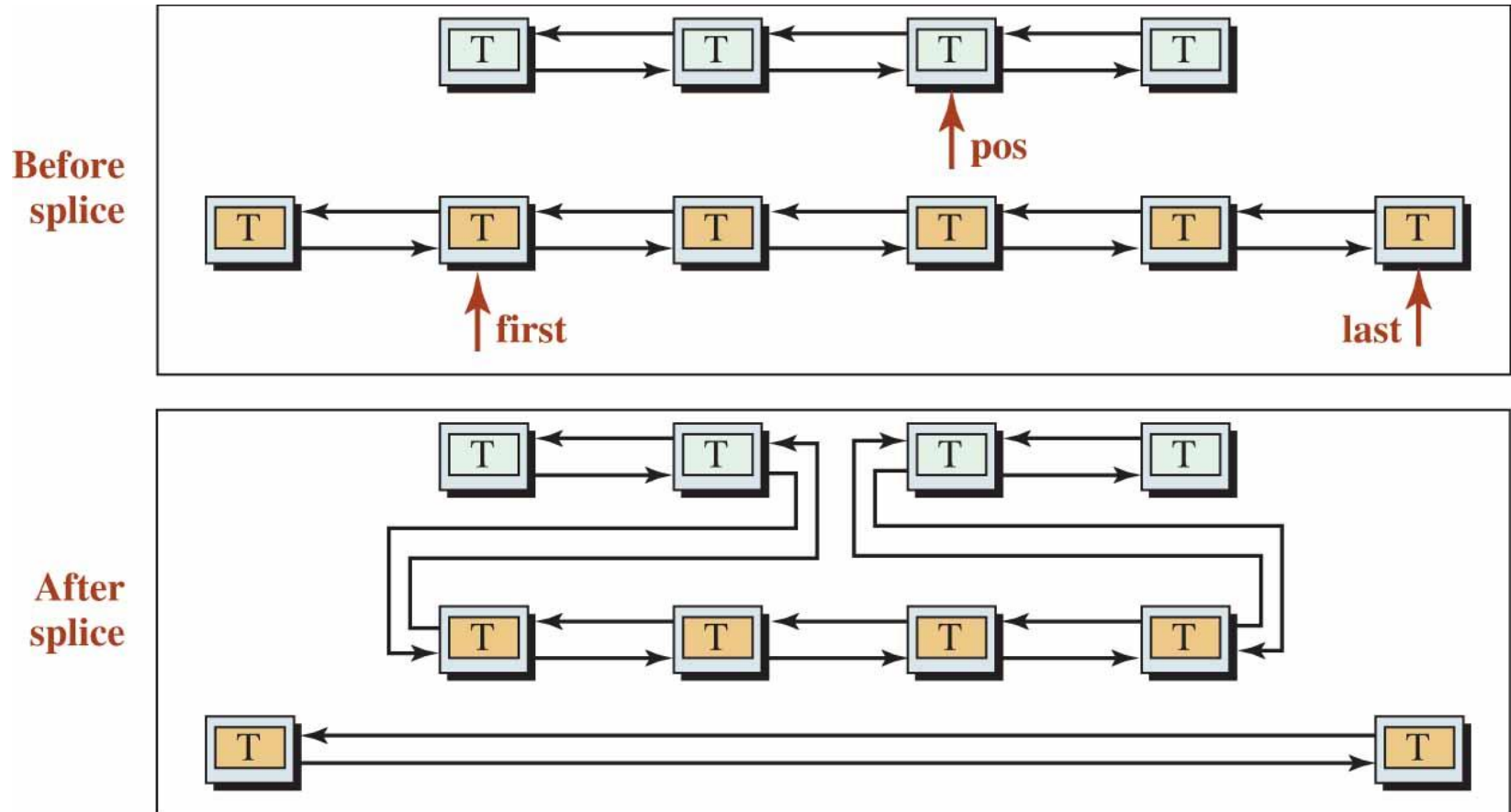
The list Class Part 6 ₍₁₎

Figure 19.10 shows the idea behind splicing.

The iterator *firstlter* defines the insertion position in the first list; the iterators *secondlter1* and *secondlter2* define the range to be moved in the second list.

The list Class Part 6 ⁽²⁾

Figure 19.10 *Splicing of two lists*



The list Class Part 9

Application Part 1.

Since the *list* class can easily grow or shrink from both ends, it has many applications.

We give some examples next.

As a simple example, let us create a list of five integers input by the user.

We then print the list in forward and reverse direction. Program 19.6 is the code.

The list Class Part 10

Application Part 2.

In Program 19.7, we repeat Program 19.2 without using *plus* or *minus* operators which are not defined for bidirectional iterators of the *list* class.

We also create a big integer class that handles any integer with any arbitrary number of digits.

We provide only the addition operation of two big integers (Program 19.8).

Printing List Elements Part 1

Program 19.6 *Printing list elements*

```
1  /*****
2  * The program that creates a list of five integers and print *
3  * in both forward and reverse direction                      *
4  *****/
5  #include <list>
6  #include <iostream>
7  using namespace std;
8
9  int main()
10 {
11     // Instantiation of a list object and declaration of a variable
12     list<int> lst;
13     int value;
14     // Inputting five integers and store then in the list
15     for (int i = 0; i < 5; i++)
16     {
17         cout << "Enter an integer: ";
18         cin >> value;
19         lst.push_back(value);
20     }
```

Printing List Elements Part 2

Program 19.6 *Printing list elements*

```
21 // Printing the list in forward direction
22 cout << "Print the list in forward direction. " << endl;
23 list<int> :: iterator iter1;
24 for(iter1 = lst.begin(); iter1 != lst.end(); iter1++)
25 {
26     cout << *iter1 << " " ;
27 }
28 cout << endl;
29 // Printing the list in backward direction
30 cout << "Print the list in reverse direction. " << endl;
31 list<int> :: reverse_iterator iter2;
32 for (iter2 = lst.rbegin(); iter2 != lst.rend(); iter2++)
33 {
34     cout << *iter2 << " " ;
35 }
36 return 0;
37 }
```

Printing List Elements Part 3

Program 19.6 *Printing list elements*

Run:

Enter an integer: 25

Enter an integer: 32

Enter an integer: 41

Enter an integer: 72

Enter an integer: 95

Print the list in forward direction.

25 32 41 72 95

Print the list in reverse direction.

95 72 41 32 25

Printing Some Elements in a List Part 1

Program 19.7 *Printing some elements in a list*

```
1  /*****
2  * Rewrite of Program 19-2 using list instead of vector      *
3  *****/
4  #include <iostream>
5  #include <list>
6  using namespace std;
7
8  int main()
9  {
10     // Instantiation of the list container and defining two iterators
11     list<int> lst;
12     list<int> :: iterator iter1;
13     list<int> :: reverse_iterator iter2;
14     // Inserting 10 integers into the list
15     for (int i = 0; i < 10; i++)
16     {
17         lst.push_back(i * 10);
18     }
19     // Moving two steps forward and two steps backward using iter1
20     cout << "Printing 40 followed by 20" << endl;
```

Printing Some Elements in a List Part 2

Program 19.7 *Printing some elements in a list*

```
21     iter1 = lst.begin();
22     iter1++;
23     iter1++;
24     iter1++;
25     iter1++;
26     cout << *iter1 << " ";
27     iter1--;
28     iter1--;
29     cout << *iter1 << endl;
30     // Moving two steps forward and two steps backward using iter2
31     cout << "Printing 50 followed by 70" << endl;
32     iter2 = lst.rbegin();
33     iter2++;
34     iter2++;
35     iter2++;
36     iter2++;
37     cout << *iter2 << " ";
38     iter2--;
39     iter2--;
40     cout << *iter2 << endl;
```

Printing Some Elements in a List Part 3

Program 19.7 *Printing some elements in a list*

```
41     return 0;  
42 }
```

Run:

Printing 40 followed by 20

40 20

Printing 50 followed by 70

50 70

Interface for the Big Integer Class Part 1

Program 19.8 The file *bigInteger.h*

```
1  /*****
2  * The interface for the big integer class *
3  *****/
4  #include <string>
5  #include <list>
6  #include <iostream>
7  #ifndef BIGINTEGER_H
8  #define BIGINTEGER_H
9  using namespace std;
10
11 class BigInteger
12 {
13     private:
14         list <int> lst;
15     public:
16         BigInteger();
17         BigInteger(string str);
18         ~BigInteger();
19         string toString();
20     friend BigInteger operator+(BigInteger first, BigInteger second);
```

Interface for the Big Integer Class Part 2

Program 19.8 *The file BigInteger.h*

```
21 };  
22 #endif
```

Implementation File For The BigInteger Class Part 1

Program 19.9 The file *bigInteger.cpp*

```
1  /*****
2   * The implementation file for the BigInteger class      *
3   *****/
4  #include <iostream>
5  #include <string>
6  #include <iomanip>
7  #include "bigInteger.h"
8  using namespace std;
9
10 // Default constructor
11 BigInteger :: BigInteger()
12 :lst (list<int>())
13 {
14 }
15 // Parameter constructor
16 BigInteger :: BigInteger(string str)
17 :lst(list <int>())
18 {
19     for (int i = 0; i < str.length(); i++)
20     {
```

Implementation File For The BigInteger Class Part 2

Program 19.9 The file *bigInteger.cpp*

```
21         int num = str[i] - 48;
22         lst.push_back(num);
23     }
24 }
25 // Destructor
26 BigInteger :: ~BigInteger()
27 {
28 }
29 // Changing a list to a string for printing
30 string BigInteger :: toString()
31 {
32     string strg;
33     list<int> :: iterator iter;
34     iter = lst.begin();
35     while (iter != lst.end())
36     {
37         strg.append(1, *iter + 48);
38         iter++;
39     }
40     return strg;
```

Implementation File For The BigInteger Class Part 3

Program 19.9 The file *bigInteger.cpp*

```
41 }
42 // Friend operator+
43 BigInteger operator+(BigInteger first, BigInteger second)
44 {
45     list<int> :: reverse_iterator iter1;
46     list<int> :: reverse_iterator iter2;
47     BigInteger result;
48     int num1, num2, sum;
49     int carry = 0;
50     iter1 = first.lst.rbegin();
51     iter2 = second.lst.rbegin();
52     while ( (iter1 != first.lst.rend()) && (iter2 != second.lst.rend()) )
53     {
54         num1 = *iter1;
55         num2 = *iter2;
56         sum = (num1 + num2 + carry) % 10;
57         carry = (num1 + num2 + carry) / 10;
58         result.lst.push_front(sum);
59         iter1++;
60         iter2++;
    }
```


Implementation File For The BigInteger Class Part 4

Program 19.9 The file *bigInteger.cpp*

```
61     }
62     while ( (iter1 != first.lst.rend()))
63     {
64         num1 = *iter1;
65         sum = (num1 + carry) % 10;
66         carry = (num1 + carry) / 10;
67         result.lst.push_front (sum);
68         iter1++;
69     }
70     while ( (iter2 != second.lst.rend()))
71     {
72         num2 = *iter2;
73         sum = (num2 + carry) % 10;
74         carry = (num2 + carry) / 10;
75         result.lst.push_front (sum);
76         iter2++;
77     }
78     if (carry == 1)
79     {
80         result.lst.push_front (carry);
```

Implementation File For The BigInteger Class Part 6

Program 19.9 *The file BigInteger.cpp*

```
81     }  
82     return result;  
83 }
```

Application File to Test the BigInteger Class Part 1

Program 19.10 *The application file (app.cpp)*

```
1  /*****
2  * Application file to test the BigInteger class          *
3  *****/
4  #include <iostream>
5  #include <iomanip>
6  #include "BigInteger.h"
7  using namespace std;
8
9  int main()
10 {
11     // Inputting two strings
12     string strg1, strg2;
13     cout << "Enter the first big integer: ";
14     cin >> strg1;
15     cout << "Enter the second big integer: ";
16     cin >> strg2;
17     // Creation of two objects of type BigInteger
18     BigInteger first (strg1);
19     BigInteger second (strg2);
20     // Adding two big integers and storing the result in the third object
```

Application File to Test the BigInteger Class Part 2

Program 19.10 *The application file (app.cpp)*

```
21     BigInteger result = first + second;
22     // Changing big integers to strings
23     strg1 = first.toString();
24     strg2 = second.toString();
25     string strg3 = result.toString();
26     string dashes(strg3.length(), '-');
27     // Printing the result
28     cout << "First big integer " << setw(strg3.length());
29     cout << right << strg1 << " + " << endl;
30     cout << "Second big integer" << setw(strg3.length());
31     cout << right << strg2 << endl;
32     cout << " " << dashes << endl;
33     cout << "Result " << setw(strg3.length());
34     cout << right << strg3 << endl;
35     return 0;
36 }
```

Application File to Test the BigInteger Class Part 3

Program 19.10 *The application file (app.cpp)*

Run:

Enter the first big integer:346786543098762378

Enter the second big integer:78654329876

First big integer	346786543098762378	+
Second big integer	78654329876	

Result	346786621753092254	

Run:

Enter the first big integer: 3478654212345690

Enter the second big integer:7654329876534567

First big integer	3478654212345690	+
Second big integer	7654329876534567	

Result	11132984088880257	

Application File to Test the BigInteger Class Part 4

Program 19.10 *The application file (app.cpp)*

Run:

Enter the first big integer: 356709876567

Enter the second big integer:19283848484848987654356

First big integer	356709876567 +
-------------------	----------------

Second big integer	19283848484848987654356
--------------------	-------------------------

Result	19283848485205697530923
--------	-------------------------

CONTAINER ADAPTERS

The standard library also defines three container adapters that have a smaller interface for easier use. The container adapters defined in the library are *stack*, *queue*, and *priority_queue*.

A very important point we need to remember is that, unlike containers, we cannot apply the algorithms defined in the library to container adapters because they lack iterators; they do not provide the member functions, such as *begin* and *end*, to create iterators.

Container adapters cannot be used with algorithms because of no supports for iterators.

Public Interface Part 1 ₍₂₎

Table 19.3 give a general interface.

The abbreviation **Ad** can be replaced with one of the three adapters (*stack*, *queue*, or *priority_queue*) when the corresponding column is ticked.

Each of these adapters is a template class in which **T** defines the type of the element in the container.

The abbreviations are **S** (*stack*), **Q** (*queue*), and **P** (*priority queue*).

Public Interface Part 2 ₍₂₎

Table 19.3 *Public interface for the container adapters classes*

Constructors	S	Q	P
Ad <T> :: Ad ()	✓	✓	✓
Checking size and emptiness			
size_type Ad<T> :: size() const	✓	✓	✓
bool Ad <T> :: empty() const	✓	✓	✓
Accessing elements			
T& Ad <T> :: front()		✓	
T& Ad <T> :: back()		✓	
T& Ad <T> :: top()	✓		✓

Public Interface Part 3 ₍₂₎

Table 19.3 *Public interface for the container adapters classes*

Insertion	S	Q	P
<code>void Ad <T> :: push(const T& elem)</code>	√	√	√
Erasure			
<code>void Ad <T> :: pop()</code>	√	√	√

Note that the interface for the *stack* and *priority_queue* is similar although their purpose and applications are different.

We can only use the *top()* member function to access an element in a *stack* or *priority_queue*.

On the other hand, we can use the *front()* and *back()* member functions to access the front or the back element in a *queue*.

The stack Class Part 1

The *stack* class is a container adapter class that is designed for three simple operations: *push*, *pop* and *top*.

The stack class is designed for insertion to and erasure from one end (the top).

It is also referred to as a *last-in, first-out* or LIFO structure because the last item pushed into the stack is the first item popped from the stack.

The stack Class Part 2

Operations

The stack interface creates an empty constructor. We can check the size and emptiness of a stack.

The only member function that we can use to access the elements in the stack is the *top()* member function.

The *push()* and *pop()* operations are used to insert into and erase an element from the stack respectively.

The stack Class Part 3

Application

The last-in first-out nature of a stack makes it ideal for applications in which the first object we need to use in a collection of objects is the last one that we created.

In Program 19.11, we change a decimal number to a hexadecimal. Although the code is similar to previous one, we use the available stack class.

Covert a Decimal to Hexadecimal Part 1

Program 19.11 Converting a decimal to hexadecimal

```
1  /*****
2   * Using a stack to change a decimal number to hexadecimal      *
3   *****/
4  #include <stack>
5  #include <iostream>
6  using namespace std;
7
8  int main()
9  {
10     // Instantiation of a stack
11     stack <char> stk;
12     // Creation of two strings and a declaration of a variable
13     string converter("0123456789ABCDEF");
14     string hexadecimal;
15     int decimal;
16     // Inputting a decimal number
17     do
18     {
19         cout << "Enter a positive integer: ";
20         cin >> decimal;
```

Covert a Decimal to Hexadecimal Part 2

Program 19.11 Converting a decimal to hexadecimal

```
21     } while (decimal <= 0);  
22     // Creation of hexadecimal characters and push them into stack  
23     while (decimal != 0)  
24     {  
25         stk.push(converter [decimal % 16]);  
26         decimal = decimal / 16;  
27     }  
28     // Popping characters from stack and pushing into hex string  
29     while (!stk.empty())  
30     {  
31         hexadecimal.push_back(stk.top());  
32         stk.pop();  
33     }  
34     cout << "The hexadecimal number : " << hexadecimal;  
35     return 0;  
36 }
```

Covert a Decimal to Hexadecimal Part 3

Program 19.11 Converting a decimal to hexadecimal

Run:

Enter a positive integer: 182

The hexadecimal number : B6

Run:

Enter a positive integer: 1234

The hexadecimal number : 4D2

Run:

Enter a positive integer: 23

The hexadecimal number : 17

The stack Class Part 4

Another example is found in compiler design.

The compiler needs to verify that the parentheses used in an expression are properly paired; every open parenthesis must be paired with a close parenthesis.

The following shows some examples of paired and unpaired parentheses.

Paired parentheses	Unpaired parentheses
$(2 + 5) * (3 - 4)$	$10 - (5 * (6 + 7)$
$4 + 5 * (6 + 7)$	$8 - (5 * 6 + 7) + 4)$

The stack Class Part 5

Note that we do not mean that the number of opening and closing parentheses should be the same, we mean that each open parentheses needs to be paired with a closing parentheses.

For example, in the expression.

$(3 + 4 (7 + 4))) 7 (8$

the number of opening and closing parentheses are the same, but they are not paired.

Program 19.12 uses a stack to check if the parentheses are paired in an expression.

Checking Pairing of Parentheses Part 1

Program 19.12 Checking pairing of parentheses

```
1  /*****
2  * A program to check if parentheses are paired in an expression *
3  *****/
4  #include <stack>
5  #include <string>
6  #include <iostream>
7  using namespace std;
8
9  int main()
10 {
11     // Declaration of a stack, a string, and a boolean object
12     stack <char> stk;
13     string expr;
14     bool paired = true;
15     // Inputting an expression and pushing it into or popping the stack
16     cout << "Enter an expression: " ;
17     getline(cin, expr);
18     int i = 0;
19     while (i < expr.size() && paired)
20     {
```

Checking Pairing of Parentheses Part 2

Program 19.12 Checking pairing of parentheses

```
21     char next = expr[i];
22     if (next == '(')
23     {
24         stk.push(next);
25     }
26     else if (next == ')')
27     {
28         if (stk.empty())
29         {
30             paired = false; // If stack is empty here, no pairing
31         }
32         else
33         {
34             stk.pop();
35         }
36     }
37     i++;
38 }
39 // If stack is not empty at the end, parentheses are not paired
40 if (!stk.empty())
```

Checking Pairing of Parentheses Part 3

Program 19.12 Checking pairing of parentheses

```
41     {  
42         paired = false;  
43     }  
44     // Print the result  
45     if (paired)  
46     {  
47         cout << "Parentheses are paired!" << endl;  
48     }  
49     else  
50     {  
51         cout << "Parentheses are not paired!" << endl;  
52     }  
53     return 0;  
54 }
```

Run:

```
Enter an expression:3 + (4 + 7 + 6)  
Parentheses are paired!
```

Run:

```
Enter an expression:2 + (4 + (6 + 7)  
Parentheses are not paired!
```

Checking Pairing of Parentheses Part 4

Program 19.12 *Checking pairing of parentheses*

Run:

Enter an expression: $6 + 7 + (10 * 2 - 4) + 8)$

Parentheses are not paired!

The queue Class Part 1

The *queue* class, which is defined in the `<queue>` header file, is a container adapter class that is designed for three simple operations: insertion at one end, erasure from the other end, and accessing at both ends.

It is also referred to as *first-in, first-out* or FIFO structure because the first item pushed into the queue is the first item to be popped from the queue.

The queue Class Part 2

Operations

The queue interface allows us to create an empty queue. We can check the size and emptiness of a queue.

The only two member functions that we can use to access the elements in the queue are the *front()* and *back()* member functions each in two versions (constant and non-constant).

The *push()* and *pop()* operations are used to insert in and erase an element from the queue respectively.

Application

The first-in, first-out nature of a queue makes it the best application for entities waiting to be served by servers.

In Program 19.13, we redesign the program that categorizes a list of donation but use the STL *queue* class instead of user-defined.

A Program to Group Charity Donations

Part 1

Program 19.13 *Grouping charity donations*

```
1  /*****
2  * Using a queue to categorize a list of donations          *
3  *****/
4  #include <queue>
5  #include <cstdlib>
6  #include <ctime>
7  #include <iostream>
8  using namespace std;
9
10 // Declaration of print function
11 void print(queue <int> queue);
12
13 int main()
14 {
15     // Instantiation of five queue objects and two variables
16     queue <int> queue1, queue2, queue3, queue4, queue5;
17     int num;
18     int donation;
19     // Random creation of donation values and push them in queues
20     srand(time(0));
```

A Program to Group Charity Donations

Part 2

Program 19.13 Grouping charity donations

```
21  for (int i = 0; i < 50; i++)
22  {
23      num = rand();
24      donation = num % (50 - 0 + 0) + 0;
25      switch (donation / 10)
26      {
27          case 0: queue1.push(donation);
28                  break;
29          case 1: queue2.push(donation);
30                  break;
31          case 2: queue3.push(donation);
32                  break;
33          case 3: queue4.push(donation);
34                  break;
35          case 4: queue5.push(donation);
36                  break;
37      }
38  }
39  // Printing the donations in each group
40  cout << "Donations between 00 and 09: ";
```

A Program to Group Charity Donations

Part 3

Program 19.13 Grouping charity donations

```
41     print(queue1);
42     cout << "Donations between 10 and 19: ";
43     print(queue2);
44     cout << "Donations between 20 and 29: ";
45     print(queue3);
46     cout << "Donations between 30 and 39: ";
47     print(queue4);
48     cout << "Donations between 40 and 49: ";
49     print(queue5);
50     return 0;
51 }
52 // Definition of the print function
53 void print(queue <int> queue)
54 {
55     while (!queue.empty())
56     {
57         cout << queue.front() << " ";
58         queue.pop();
59     }
60     cout << endl;
```

A Program to Group Charity Donations

Part 4

Program 19.13 *Grouping charity donations*

```
61 }
```

Run:

Donations between 00 and 09: 3 1 3 7 9 1 8 8 9 7 4 3

Donations between 10 and 19: 18 14 13 15 16 15

Donations between 20 and 29: 23 26 28 29 23 21 26 21 22 27 28 23

Donations between 30 and 39: 32 31 39 39 39 34 30 38 30 36 32

Donations between 40 and 49: 41 49 49 43 47 43 45 46 41

The priority_queue Class Part 1

The ***priority_queue*** class defined in the `<queue>` header file is a container adapter in which each element has a priority level.

The elements can be inserted into the ***priority_queue*** in any order; the elements are retrieved based on their priority.

In other words, the front element is the element with the largest priority in the container.

The priority_queue Class Part 2

Operations

The interface of a *priority_queue* is given in Table 19.2.

Like a *queue*, we *push* at the *back* and *pop* in the *front*, but the access is limited only to the *front*, which is called *top()*, like a stack; we cannot access elements at the *back*.

An implementation difference that we can see between a *queue* and a *priority_queue* is that the *priority_queue* does not support relational operators (see Table 19.2).

In other words, two priority queues cannot be compared.

The priority_queue Class Part 3

Application

The *priority_queue* is the only structure available when we need to access the elements in the order they enter the container, but give priority to those that need faster access. For example, in a restaurant people with reservations have priority.

Although we can use a *pair* class, a *tuple* class, or a user-created class to define the type of the elements in a *priority queue*, they require that we define the comparison for the data type or use a *function object* to do so.

We leave this until we learn about *function objects*. Program 19.14 shows a very simple example of a *priority queue* application of type integer that reorders the elements according to their integer values (Program 19.14).

Using a Priority Queue Part 1

Program 19.14 Using a priority queue

```
1  /*****
2  * A program to test the priority_queue class
3  *****/
4  #include <queue>
5  #include <iostream>
6  using namespace std;
7
8  int main()
9  {
10     // Create a priority_queue object
11     priority_queue <int> line;
12     // Push some elements
13     line.push(4);
14     line.push(7);
15     line.push(2);
16     line.push(6);
17     line.push(7);
18     line.push(8);
19     line.push(2);
20     // Print the elements according to their priorities
```


Using a Priority Queue Part 2

Program 19.14 *Using a priority queue*

```
21 while (!line.empty ())
22 {
23     cout << line.top() << " ";
24     line.pop();
25 }
26 return 0;
27 }
```

Run:

8 7 7 6 4 2 2

ASSOCIATIVE CONTAINERS

Elements in an *associative container* are stored and retrieved by a *key*.

To access an element in an associate container, we need to use the *key* of the element.

Associate containers are divided into two classes (*set* and *map*).

- A *set* is an associative container in which the *key* and *value* are the same data item.
- A *map* is an associative container in which the *key* and the *value* are separate data items.

Public Interface Part 4₍₂₎

Table 19.4 gives a general public interface for the *set* and *map* classes.

The abbreviation **AS** must be replaced with one of the two *associative containers* (*set* or *map*) when the corresponding column (**S** or **M**) is ticked. **S** means *set* and **M** means *map*/.

Table 19.4 *The public interface for the set and map classes*

Constructors, assignment, and destructor	S	M
AS <...> :: AS ()	√	√
AS <...> :: AS (<i>const_iterator</i> first, <i>const_iterator</i> last)	√	√
AS <...> :: AS (const AS <...>& s)	√	√
AS <...>& AS <...> :: operator =(const AS <...> & right)	√	√
AS <...> :: ~ AS ()	√	√
Controlling size	S	M
<i>size_type</i> AS <...> :: size()	√	√
<i>size_type</i> AS <...> :: max_size()	√	√
<i>bool</i> AS <...> :: empty()	√	√

Public Interface Part 5 ₍₂₎

Table 19.4 *The public interface for the set and map classes*

Iterators	S	M
iterator AS <...> :: begin()	✓	✓
iterator AS <...> :: end()	✓	✓
reverse_iterator AS <...> :: rbegin()	✓	✓
reverse_iterator AS <...> :: rend()	✓	✓
Accessing	S	M
T& AS <...> :: operator [](const K& k) const		✓
Searching	S	M
size_type AS <...> :: count(const K& k)	✓	✓
iterator AS <...> :: find(const K& k) const	✓	✓
iterator AS <...> :: lower_bound(const K& k) const	✓	✓
iterator AS <...> :: upper_bound(const K& k) const	✓	✓
pair <iterator, iterator> AS <...> :: equal_range(const K& k) const	✓	✓
Insertion	S	M
pair <iterator, bool> AS <...> :: insert(const K& k)	✓	
pair <iterator, bool> AS <...> :: insert(const pair < const K, T > & p)		✓
iterator AS <...> :: insert(iterator hintpos, const K& element)	✓	✓
void AS <...> :: insert(InputIterator first, InputIterator last)	✓	✓

Public Interface Part 6₍₂₎

Table 19.4 *The public interface for the set and map classes*

Erasure	S	M
<code>size_type AS <...> :: erase(const K& k)</code>	✓	✓
<code>void AS <...> :: erase(iterator pos)</code>	✓	✓
<code>void AS <...> :: erase(iterator first, iterator last)</code>	✓	✓
<code>void AS <...> :: clear()</code>	✓	✓
Swapping	S	M
<code>void AS <...> :: swap(AS <...>& v)</code>		✓
Global Functions	S	M
<code>bool operator oper(const AS <...> left, const AS <...> right)</code>	✓	✓
<code>void swap(AS <...>& c1, AS <...>& c2)</code>	✓	✓

Public Interface Part 7

To discuss the class *set* and *map*, we need to introduce a library *struct* named *pair* defined in the `<utility>` header.

A *pair* defines a template *struct* with two template data members possibly of different types.

The elements are called *first* and *second* respectively as shown below:

```
template <typename T1, typename T2>
struct pair
{
    T1 first;
    T2 second;
};
```

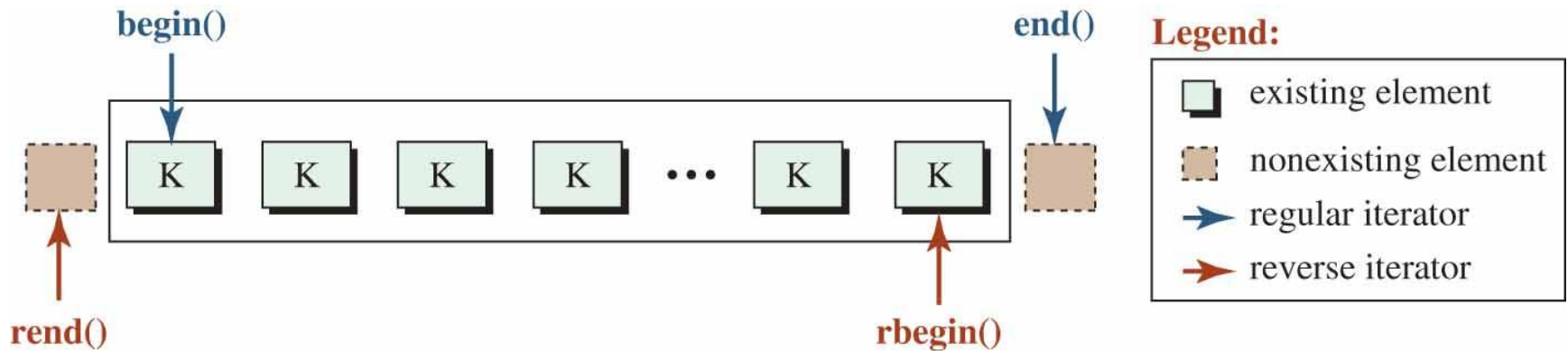
Set Part 1

In a *set*, each element in the container stores one template value which is referred to as the *key*.

The elements are sorted in ascending order and duplicates are not allowed. Figure 19.11 shows an example of a *set*.

Set Part 2

Figure 19.11 *A set*



Notes:

The keys are unique in a set.

The objects are in a nonlinear relationship with each other.

The *set* class uses bidirectional iterators.

Although the objects appear to be one after another to the user, they are not linearly connected.

Set Part 3₍₁₎

Operations

The following are operations that can be used with a *set*.

Constructors, Destructor, and Assignment.

We can create a new set using a default constructor (empty set).

We can also create a new set using a parameter constructor by copying the elements of another set in which the range is indicated by [first, last).

We can also create a set by using a copy constructor, or an assignment operator.

The following shows how to create a set of integers.

Set Part 3₍₂₎

```
set <type> set1;           // Create an empty set
set <type>(pos1, pos2) set2; // Create a set from part of another set
set <type>(set2) set3;      // Create a set from another set
set4 = set3;               // Assignment
```

Controlling Size.

There are three member functions that we can be used to check size, maximum size, and emptiness.

They are the same as discussed for sequence containers.

Set Part 4

Iterators

The set class uses bidirectional (not random-access) iterators because it is normally implemented as a non-linear linked-list.

It provides the same eight internal iterators as the sequence containers in which four are constant and four non-constant.

```
set1.begin()    // A regular iterator to the first element
set1.end()      // A regular iterator to the element after the last
set1.rbegin()   // A reverse iterator to the last element
set1.rend()     // A reverse iterator to the element before the last
```

Set Part 5

Searching

Since the elements in a set are sorted, searching is possible and efficient. There are four members for searching as shown below:

```
set1.count(k)           // Returns number of elements equal to k
set1.find(k)            // Returns an iterator pointing to the first k found
set1.lower_bound(k)     // Returns the first position where k can be inserted
set1.upper_bound(k)     // Returns the last position where k can be inserted
set1.equal_range(k)     // Combination of lower and upper bound
```

Set Part 6₍₁₎

Insertion

There are no push members to insert an element in a set.

Insertion must be done using the key or through iterators.

```
set1.insert(k)           // Returns a regular iterator to first element
set1.insert(hint, k)      // Returns a regular iterator after the last
set1.insert(pos1, pos2)   // Returns an iterator to the last element
```

Set Part 6₍₂₎

Erasure

There are no pop members to delete an element from a set.

Deletion must be done through the value or iterator.

```
set1.erase(k)           // Erases k and returns a pair <pos, bool>
set1.erase(pos)         // Erases an element pointed to by pos
set1.erase(first, last) // Erases elements in the range [first, last)
set1.clear()            // Returns an iterator after the last
```

Other Operations.

Just like the sequence containers, the *swap()* operation is defined for sets.

Also we can compare two sets with relational operators.

Set Part 7

Applications

We show two simple applications of sets in this section.

In the first example we use a program to show we can create a set of integers, print in the ascending and descending order and print an integer before a given integer and an integer after a given integer (Program 19.15).

In the second example, we create a student set and overload the less than operator for the Student class to be able to list students sorted according to their student identification (Program 19.16, 19.17, and 19.18).

Program to Handle a Set of Integers Part 1

Program 19.15 Handling a set of integers

```
1  /*****
2  * Using a set to create a sorted container of integers      *
3  *****/
4  #include <set>
5  #include <iostream>
6  #include <iomanip>
7  using namespace std;
8
9  int main()
10 {
11     // Create an empty set of integers
12     set <int> st;
13     // Insert some keys in the set (with duplicates)
14     st.insert(47);
15     st.insert(18);
16     st.insert(12);
17     st.insert(24);
18     st.insert(52);
19     st.insert(20);
20     st.insert(24);
```


Program to Handle a Set of Integers Part 2

Program 19.15 Handling a set of integers

```
21     st.insert(92);
22     st.insert(53);
23     st.insert(77);
24     st.insert(98);
25     st.insert(87);
26     // Print the set elements from smallest to largest
27     cout << "Printing set elements from smallest to largest." << endl;
28     set<int> :: iterator iter;
29     for (iter = st.begin(); iter != st.end(); iter++)
30     {
31         cout << setw(4) << *iter;
32     }
33     cout << endl << endl;
34     // Print the set elements from largest to smallest
35     cout << "Printing set elements from largest to smallest." << endl;;
36     set<int> :: reverse_iterator riter;
37     for (riter = st.rbegin(); riter != st.rend(); riter++)
38     {
39         cout << setw(4) << *riter;
40     }
```

Program to Handle a Set of Integers Part 3

Program 19.15 Handling a set of integers

```
41      cout << endl << endl;
42      // Print the element after 52
43      set <int> :: iterator iter1 = st.find(52);
44      iter1++;
45      cout << "Element after 52: " << *iter1 << endl;
46      // Print the element before 20
47      set <int> :: iterator iter2 = st.find (20);
48
49      iter2--;
50      cout << "Element before 20: " << *iter2 << endl;
51      return 0;
52  }
```

Run:

Printing set elements from smallest to largest.

12 18 20 24 47 52 53 77 87 92 98

Printing set elements from largest to smallest.

98 92 87 77 53 52 47 24 20 18 12

Element after 52: 53

Element before 20: 18

Interface File for the Student Class Part 1

Program 19.16 The interface file (student.h)

```
1  /*****
2  * The interface file for the Student class *
3  *****/
4  #ifndef STUDENT_H
5  #define STUDENT_H
6  #include <string>
7  #include <set>
8  #include <iostream>
9  #include <iomanip>
10 using namespace std;
11
12 class Student
13 {
14     private:
15         int identity ;
16         string name;
17         double gpa;
18     public:
19         Student(int identity, string name, double gpa);
20         ~Student();
```

Interface File for the Student Class Part 2

Program 19.16 *The interface file (student.h)*

```
21     void print() const;  
22     bool friend operator< (const Student& left, const Student& right);  
23 };  
24 #endif
```

Implementation File for the Student Class

Part 1

Program 19.17 The implementation file (student.cpp)

```
1  /*****
2  * The implementation file for the Student class      *
3  *****/
4  #include "student.h"
5
6  // Constructor
7  Student :: Student(int id, string nm, double gp)
8  : identity (id), name (nm), gpa (gp)
9  {
10 }
11 // Destructor
12 Student :: ~Student()
13 {
14 }
15 // Print member function
16 void Student :: print() const
17 {
18     cout << setw(3) << right << identity << " " ;
19     cout << setw(12) << left << name << " " ;
20     cout << setw(6) << right << showpoint << setprecision(3) ;
```

Implementation File for the Student Class

Part 2

Program 19.17 *The implementation file (student.cpp)*

```
21     cout << gpa << " " << endl;
22 }
23 // Friend less-than operator
24 bool operator<(const Student& left, const Student& right)
25 {
26     return (left.identity < right.identity) ;
27 }
```

Application File for the Student Class

Part 1 ₍₁₎

Program 19.18 *The application file (app.cpp)*

```
1  /*****
2  * The application file for the Student class          *
3  *****/
4  #include "student.h"
5
6  int main()
7  {
8      // Creating six instance of the Student class
9      Student student1(120, "George", 3.78);
10     Student student2(185, "Mary", 3.95);
11     Student student3(110, "Richard", 4.00);
12     Student student4(245, "Alen", 3.70);
13     Student student5(172, "John", 3.00);
14     Student student6(195, "Lucie", 3.80);
15     // Insert the above six object into the set
16     set <Student> stdSet ;
17     stdSet.insert(student1);
18     stdSet.insert(student2);
19     stdSet.insert(student3);
20     stdSet.insert(student4);
```

Application File for the Student Class

Part 1 ₍₂₎

Program 19.18 *The application file (app.cpp)*

```
21     stdSet.insert(student5);
22     stdSet.insert(student6);
23     // Printing the contents of student objects
24     set <Student> :: iterator iter;
25     for (iter = stdSet.begin(); iter != stdSet.end(); iter++)
26     {
27         iter -> print();
28     }
29     return 0;
30 }
```

Run:

```
110  Richard  4.00
120  George   3.78
172  John     3.00
185  Mary     3.95
195  Lucie    3.80
245  Alen     3.70
```


Map Part 1

A *map*, which is also called a *table*, a *dictionary*, or an *associate* array, is defined in the `<map>` header file.

It is a container that stores a template *pair* of *key* and *value*.

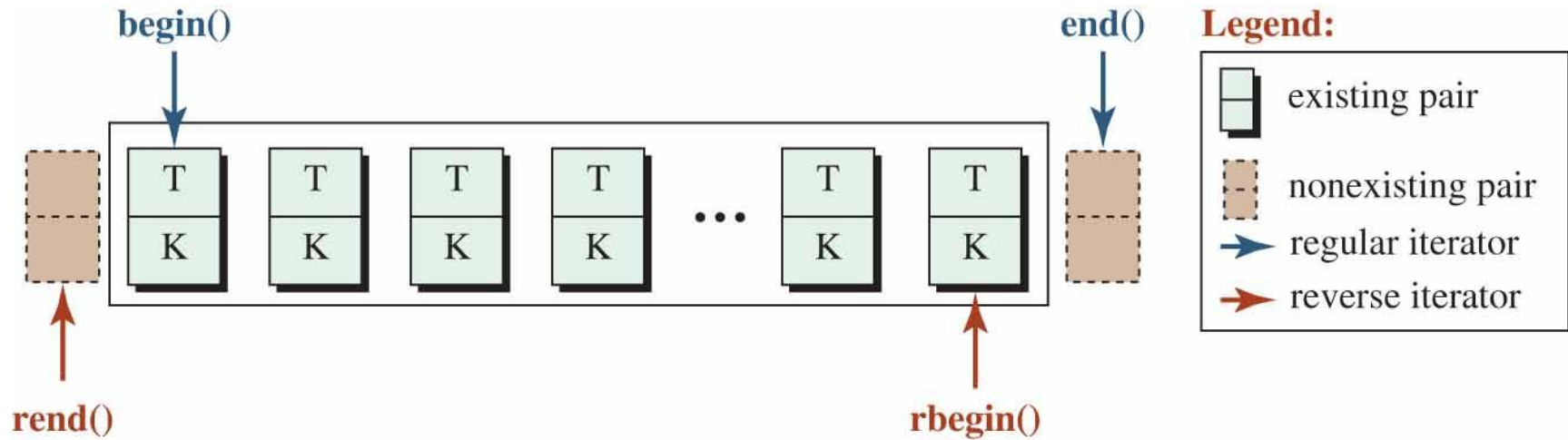
The elements are sorted in ascending order based on the key.

In a map, the keys are unique.

Figure 19.12 shows an example of a map.

Map Part 2

Figure 19.12 *A map*



Notes:

The keys are unique in a map.
The objects are in nonlinear relationship to each other.
The *map* class uses bidirectional iterators.

Map Part 3 ₍₁₎

Operations Part 1.

The operations defined for the map class are very similar to the ones for the set class.

The main difference is the ability to access the elements in a map using the operator [].

This operator makes a map object look like an array in which the index is a key value instead of an integer.

Map Part 3 ₍₂₎

Operations Part 2.

In other words, if we know the value of the key in an element, we can access the element using the expression `map[key]`.

However, we need to be aware that the operator `[ ]` does not act like the one used for an array or a vector or a deque.

It is just a notation to create a pair of key-value. A map uses only bidirectional iterators and cannot jump from one pair to another using the `+` or `-` operator.

Map Part 4

Application

Among many applications, we mention two.

In the first, we create a *map* of student-scores in which each element is a pair of student name and student score in a test (Program 19.19).

In the second, we create a program that finds the frequency of words in a string.

We use a *map* object named *freq* in which the key is the word encountered in the text and the value is the frequency of words.

Note that we have used (ctrl + Z) character to signal the end of the input file (Program 19.20).

Student Table (name and score) Part 1 ⁽¹⁾

Program 19.19 A student table (name and score)

```
1  /*****
2  * A program to create five pairs of student and score in a map *
3  *****/
4  #include <map>
5  #include <iostream>
6  #include <iomanip>
7  #include <utility>
8  using namespace std;
9
10 int main()
11 {
12     // Creation of a map and the corresponding iterator
13     map <string, int > scores;
14     map <string, int > :: iterator iter;
15     // Inputting student name and score in the map
16     scores ["John"] = 52;
17     scores ["George"] = 71;
18     scores ["Mary"] = 88;
19     scores ["Lucie"] = 98;
20     scores ["Robert"] = 77;
```

Student Table (name and score) Part 1 ₍₂₎

Program 19.19 *A student table (name and score)*

```
21 // Printing the names and score sorted on names
22     cout << "Students names and scores" << endl;
23     for (iter = scores.begin (); iter != scores.end(); iter++)
24     {
25         cout << setw(10) << left << iter -> first << " ";
26         cout << setw(4) << iter -> second << endl;
27     }
28     return 0;
29 }
```

Run:

Students names and scores

George	71
John	52
Lucie	98
Mary	88
Robert	77

Frequency of Words in Text Part 1

Program 19.20 *Frequency of words in text*

```
1  /*****
2  * A program to count the number of words in a text      *
3  *****/
4  #include <map>
5  #include <string>
6  #include <iomanip>
7  #include <iostream>
8  using namespace std;
9
10 int main()
11 {
12     // Declaration of map, iterator, and a string
13     map <string, int > freq;
14     map <string, int > :: iterator iter;
15     string word;
16     // Reading and storing words in the map
17     cout << "Enter a sentence to be parsed: " << endl;
18     while (cin >> word)
19     {
20         ++freq [word];
```


Frequency of Words in Text Part 2

Program 19.20 *Frequency of words in text*

```
21     }
22     // Printing the words and their frequency
23     for (iter = freq.begin(); iter != freq.end(); iter++)
24     {
25         cout << left << setw(10) << iter -> first << iter -> second << endl;
26     }
27     return 0;
28 }
```

Frequency of Words in Text Part 3

Program 19.20 *Frequency of words in text*

Run:

Enter a sentence to be parsed:

we are in the world of this and that and this and that

^Z

and	3
-----	---

are	1
-----	---

in	1
----	---

of	1
----	---

that	2
------	---

the	1
-----	---

this	2
------	---

we	1
----	---

world	1
-------	---

Map Part 6

Note that we type all words in lower case to make easier to see that the keys are sorted.

The words and frequency form an associative array, an array whose index does not have to be integers (Figure 19.13).

Figure 19.13 *A map serving as an associative array*

3	1	1	1	2	1	2	1	1
"and"	"are"	"in"	"of"	"that"	"the"	"this"	"we"	"world"

An associative array with string indexes

USING FUNCTIONS

We can use library algorithms or define our own algorithms.

In each case, an algorithm applies an operation to a number of elements in the container.

The question is how can this operation be defined as a parameter in the algorithm.

It can be done in two ways: a pointer to function or a function object (*functor*). We discuss each approach next.

Pointer to Function Part 1

We know that the definition of a function is stored in memory. Every entity that is stored in memory has an address.

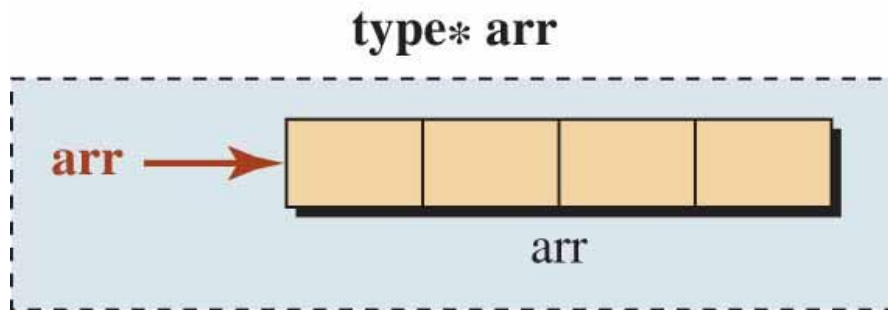
In fact, the name of a function is a pointer to the first byte of memory where the function is stored just as the name of an array is a pointer to the first element of an array.

Figure 19.14 shows how the name of an array and the name of a function are pointers.

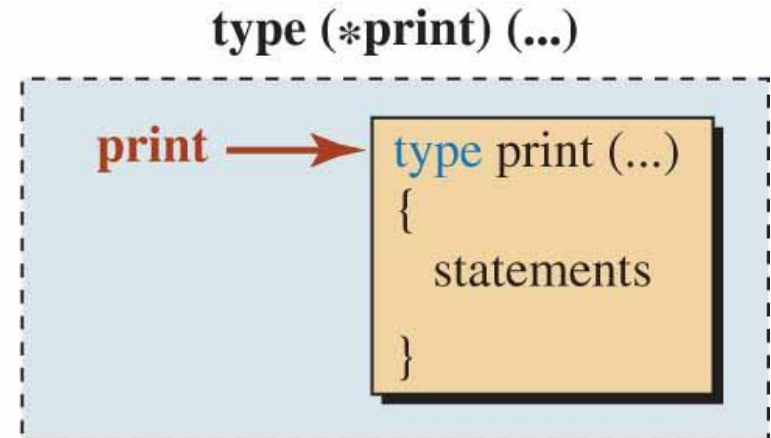
The figure also shows how a pointer to an array and a pointer to a function are declared.

Pointer to Function Part 2

Figure 19.14 *Pointer to an array and pointer to a function*



A pointer to an array



A pointer to a function

Pointer to Function Part 3

Program 19.21 is a trivial example of a pointer to function. We want to use a function, *fun*, that uses another function, *print*, as an argument.

We define both functions. In *main*, we call *fun* two times, which calls the *print* function in each call.

Note that the definition of *fun* does not define which function is to be called as the second parameter.

The calling statement passes a pointer to the *print* function (*print* is a pointer to the beginning of the *print* function).

Using Pointer to Functions Part 1 ⁽¹⁾

Program 19.21 *Using pointer to functions*

```
1  /*****
2   * Using functions that call another function          *
3   *****/
4  #include <iostream>
5  using namespace std;
6
7  // Definition of print function
8  void print (int value)
9  {
10     cout << value << endl;
11 }
12 // Definition of fun function
13 void fun (int x, void(*f)(int))
14 {
15     f(x);
16 }
17
18 int main()
19 {
20     fun(24, print); // Calling function fun
```


Using Pointer to Functions Part 1 ₍₂₎

Program 19.21 *Using pointer to functions*

```
21 fun(88, print); // Calling function fun
22 return 0;
23 }
```

Run:

24

88

Pointer to Function Part 4 ₍₁₎

The declaration of a pointer to function, **void (*f)(int)**, may look unusual, but the good news is that most algorithms in STL have already defined functions that use pointer to functions (such as *print*) in the above example.

We do not have to declare them. In other words, if an STL algorithm uses a pointer to function, we only need to give the name of the function in the call.

Pointer to Function Part 4 ₍₂₎

We use the STL generic algorithm, *for_each*, that applies a function to a range of items in a container.

The algorithm applies the function defined as the third parameter to the range [first, last) as shown in Program 19.22.

The algorithm defines that the third algorithm must be a pointer to a function.

We pass the name *print* and then we define a function named *print* with one argument.

The print function takes the value of its argument from the iterator (*iterator).

Using for_each With a User-defined Function Part 1

Program 19.22 Using for_each with a user-defined function

```
1  /*****
2   * Applying a function to all elements in a range using the      *
3   * algorithm for_each in the library                             *
4   *****/
5  #include <vector>
6  #include <algorithm>
7  #include <iostream>
8  using namespace std;
9
10 // Definition of the print function
11 void print(int value)
12 {
13     cout << value << " ";
14 }
15
16 int main()
17 {
18     // Instantiation of a vector object and storing three values
19     vector <int> vec;
20     vec.push_back(24);
```

Using for_each With a User-defined Function Part 2

Program 19.22 Using for_each with a user-defined function

```
21     vec.push_back(42);  
22     vec.push_back(73);  
23     // Using a print function to print the value of each element  
24     for_each(vec.begin(), vec.end(), print);  
25     return 0;  
26 }
```

Run:

24 42 73

Function Objects (Functors) Part 1

In a previous chapter we learned that we can overload the *function operator*, a pair of parentheses, to allow us to create a *function object* (sometimes called a *functor*).

Figure 19.15 shows how we write a regular function and a function object.

Figure 19.15 *A regular function versus a function object*

**Regular
function**

```
void sample (...)  
{  
    statements  
}
```

**Function
object**

```
class sample  
{  
    public :  
        void operator () (...)  
        {  
            statements  
        }  
};
```

Function Objects (Functors) Part 2

Program 19.23 shows how we can call a *functor* from another function (*main*).

We create a class named `Print`, overload the *operator()*, create an object of the class, and call the operator.

Note that calling the overloaded operator is simply calling the object instantiated from the class and inserting the arguments inside the parentheses.

In fact, we are calling *print.operator()(45)* in line 16.

Example of Using a Functor

Program 19.23 Using a functor

```
1  /*****
2   * Calling a function object to print the same value      *
3   *****/
4  #include <iostream>
5  using namespace std;
6
7  class Print
8  {
9      public:
10     void operator()(int value) {cout << value;}
11 };
12
13 int main()
14 {
15     Print print; // Instantiation of an object of type Print
16     print(45); // calling operator()
17     return 0;
18 }
```

Run:

45

Function Objects (Functors) Part 3

We can see that the definition of a function object is longer than a regular function, but a function object has advantages that justify its longer definition as shown below:

- As an object, a function object can be used wherever an object can be used. In particular, it can be passed to a function as an argument, and it can be returned from a function.**
- A function object can have a state, which means that it can hold information from one call to another.**
- We can define a class to be used as a function object and then inherit from it to create other function objects.**

Function Objects (Functors) Part 4

Function Objects in STL Algorithms.

The STL library defines many function objects in the `<functional>` header. They can be divided into unary and binary. Table 19.5 gives some of them.

Table 19.5 *Function objects in STL*

Function object	type	arity	operator
<code>negate <T></code>	arithmetic	unary	-
<code>plus <T></code>	arithmetic	binary	+
<code>minus <T></code>	arithmetic	binary	-
<code>multiplies <T></code>	arithmetic	binary	*
<code>divides <T></code>	arithmetic	binary	/
<code>modulus <T></code>	arithmetic	binary	%
<code>equal_to <T></code>	relational	binary	==
<code>not_equal_to <T></code>	relational	binary	!=
<code>greater <T></code>	relational	binary	>
<code>greater_equal <T></code>	relational	binary	>=
<code>less <T></code>	relational	binary	<
<code>less_equal <T></code>	relational	binary	<=
<code>logical_not <T></code>	logical	unary	!
<code>logical_and <T></code>	logical	binary	&&
<code>logical_or <T></code>	logical	binary	

Function Objects (Functors) Part 5 (1)

Each function in fact simulates one of the built-in operators.

However, the naming is different from what we have seen for built-in operators.

As an example, *negate* means the minus operator (unary) and *minus* means subtraction operator (binary).

Function Objects (Functors) Part 5 ₍₂₎

We discuss algorithms in the next section.

For the moment let us show how we use the *transform* algorithm to negate all elements in a vector as shown in Program 19.24.

Note that we do not see any object of type *negate* in the call to transform algorithm.

The reason is that the *transform* function directly calls the default constructor of the *negate* class (with pair of parentheses).

The algorithm then calls the *operator()* in the background and passes the value returned from dereferencing iterator to the that operator.

However, this happens at the background and hidden from the user.

Using Pointer to Function and Functor

Part 1

Program 19.24 Using a unary functor

```
1  /*****
2   * A program to use a pointer to function and a functor      *
3   *****/
4  #include <vector>
5  #include <algorithm>
6  #include <iostream>
7  #include <functional>
8  using namespace std;
9
10 // User-defined print function
11 void print(int value)
12 {
13     cout << value << " ";
14 }
15
16 int main()
17 {
18     // Creation of a vector with four nodes
19     vector <int> vec;
20     vec.push_back(24);
```

Using Pointer to Function and Functor

Part 2

Program 19.24 Using a unary functor

```
21     vec.push_back(42);
22     vec.push_back(73);
23     vec.push_back(92);
24     // Printing the node using a pointer to user-defined function
25     for_each(vec.begin(), vec.end(), print);
26     cout << endl;
27     // Negating the values of all nodes and print them again
28     transform(vec.begin(), vec.end(), vec.begin(), negate<int>());
29     for_each(vec.begin(), vec.end(), print);
30     return 0;
31 }
```

Run:

```
24 42 73 92
-24 -42 -73 -92
```

ALGORITHMS

Another piece of the STL is generic algorithms. Instead of defining these operations inside each container type, the C++ language defines template global functions that can be applied to any container type that supports the iterators required by the algorithm.

The algorithms are template global functions, but the template type does not define the type of the elements in the container; it defines the type of the iterator the algorithm itself uses.

Non-mutating Algorithms Part 1

Non-mutating algorithms, which are defined in the `<algorithm>` header file, do not change the order of the elements in the container that they are applied to.

They can be modifying or non-modifying.

The non-modifying algorithms do not change the value of elements in the container; the modifying ones can change the values if the operation is designed for that purpose.

Table 19.6 shows some of the most commonly used algorithms in this category.

Table 19.6 *Some non-mutating algorithms*

```
difference_type count(InIter first, InIter last, const T& value);  
difference_type count_if(InIter first, InIter last, Predicate pred);  
InIter find(InIter first, InIter last, const T& value);  
Function for_each(InIter first, outIter last, Function func);
```


Non-mutating Algorithms part 2

The first function counts the number of elements equal to *value*.

The second function counts the element if they meet the criteria in *pred*.

The third function finds the position of an element with a given value.

The fourth function applies the parameter *func* to the member elements in the range [*first, last*).

Testing Some Non-mutating Algorithms

Part 1

Program 19.25 *Testing non-mutating algorithms*

```
1  /*****
2   * A program to use some non-mutating algorithms          *
3   *****/
4  #include <vector>
5  #include <algorithm>
6  #include <iostream>
7  using namespace std;
8
9  // Definition of isEven
10 bool isEven(int value)
11 {
12     return (value % 2 == 0);
13 }
14 // Definition of timesTwo
15 void timesTwo(int& value)
16 {
17     value = value * 2;
18 }
19 // Definition of print
20 void print(int value)
```

Testing Some Non-mutating Algorithms

Part 2

Program 19.25 Testing non-mutating algorithms

```
21 {  
22     cout << value << " ";  
23 }  
24  
25 int main()  
26 {  
27     // Instantiation of a vector of integers  
28     vector <int> vec ;  
29     // Pushing ten values into the vector  
30     vec.push_back(17) ;  
31     vec.push_back(10) ;  
32     vec.push_back(13) ;  
33     vec.push_back(13) ;  
34     vec.push_back(18) ;  
35     vec.push_back(15) ;  
36     vec.push_back(17) ;  
37     vec.push_back(13) ;  
38     vec.push_back(13) ;  
39     vec.push_back(18) ;  
40     // Printing original values
```

Testing Some Non-mutating Algorithms

Part 3₍₁₎

Program 19.25 Testing non-mutating algorithms

```
41     cout << "Original values in vector" << endl;
42     for_each(vec.begin(), vec.end(), print);
43     cout << endl << endl;
44     // Counting number of 10's
45     cout << "Count of 10's: ";
46     cout << count(vec.begin(), vec.end(), 10);
47     cout << endl << endl;
48     // Counting the even values
49     cout << "Count of even values: ";
50     cout << count_if( vec.begin(), vec.end(), isEven );
51     cout << endl << endl;
52     // Doubling each value and printing vector
53     cout << "Values after multiplying by 2" << endl;
54     for_each(vec.begin(), vec.end(), timesTwo);
55     for_each(vec.begin(), vec.end(), print);
56     return 0;
57 }
```

Testing Some Non-mutating Algorithms

Part 3 ₍₂₎

Program 19.25 *Testing non-mutating algorithms*

Run:

Original values in vector

17 10 13 13 18 15 17 13 13 18

Count of 10's: 1

Count of even values: 3

Values after multiplying by 2

34 20 26 26 36 30 34 26 26 36

Mutating Algorithms Part 1

The mutating algorithms, which are defined in the `<algorithm>` header file, change the structure of the container they are applied to.

This list is long, but we show three of them in Table 19.7 (*Bditer* means bidirectional iterator and *Fwiter* means forward iterator).

Table 19.7 *Some mutating algorithms*

```
void generate(Bditer first, Bditer last, gen);  
void reverse(Bditer first, Bditer last);  
void rotate(Fwiter first, Fwiter middle, Fwiter last);  
void random_shuffle(Bditer first, Bditer last);  
outiter transform(initer first, initer second, outiter start, oper);
```

Mutating Algorithms Part 2

The first function creates a sequence with the result of running the *gen* function.

The second function reverses the order of elements in the container.

The third function rotates the elements to the left so that the middle element becomes the first element and the element before the middle element becomes the last.

The *random_shuffle* function changes the order in the container to a random order.

The transform function changes the values from the member pointed to by *first* to *second* and puts the result starting from element pointed to by *start*.

It returns a pointer to the last modified element.

Program to Test Mutating Functions

Part 1

Program 19.26 *Testing mutating functions*

```
1  /*****
2   * A program to use some mutating algorithms                      *
3   *****/
4  #include <vector>
5  #include <algorithm>
6  #include <iostream>
7  #include <iomanip>
8  using namespace std;
9
10 // Definition of print function
11 void print(int value)
12 {
13     cout << value << " ";
14 }
15
16 int main()
17 {
18     // Instantiation of a vector object
19     vector <int> vec ;
20     // Adding six values
```


Program to Test Mutating Functions

Part 2

Program 19.26 *Testing mutating functions*

```
21     vec.push_back(11);
22     vec.push_back(14);
23     vec.push_back(17);
24     vec.push_back(23);
25     vec.push_back(35);
26     vec.push_back(52);
27     // Printing original values
28     cout << "Original vector" << endl;
29     for_each(vec.begin(), vec.end(), print);
30     cout << endl << endl;
31     // Reversing the values and print the vector
32     cout << "Vector after reversing the order" << endl;
33     reverse(vec.begin(), vec.end());
34     for_each(vec.begin(), vec.end(), print);
35     cout << endl << endl;
36     // Rotate the values and print the vector
37     cout << "Vector after rotating the order" << endl;
38     rotate(vec.begin(), vec.begin() + 2, vec.end());
39     for_each(vec.begin(), vec.end(), print);
40     cout << endl << endl;
```

Program to Test Mutating Functions

Part 3

Program 19.26 Testing mutating functions

```
41 // Random shuffle the value print the vector
42 cout << "Vector after random shuffle" << endl;
43 random_shuffle(vec.begin(), vec.end());
44 for_each(vec.begin(), vec.end(), print);
45 cout << endl << endl;
46 return 0;
47 }
```

Run:

Original vector

11 14 17 23 35 52

Vector after reversing the order

52 35 23 17 14 11

Vector after rotating the order

23 17 14 11 52 35

Vector after random shuffle

23 14 35 52 17 11

Sorting and Related Algorithms Part 1

The <algorithm> header file also defines several algorithms that either sort the sequence or apply operations that are related to sorting.

We mention eight of these algorithms in Table 19.8 and give some examples.

Table 19.8 *Sorting and related algorithm*

```
void sort(RndIter first, RndIter last);
bool binary_search(FwIter first, FwIter last, const T& value);
FwIter min_element(FwIter first, FwIter last);
FwIter max_element(FwIter first, FwIter last);
OutIter set_difference(InIter first1, InIter last1, InIter first2, InIter last2,
                      OutIter result);
OutIter set_intersection(InIter first1, InIter last1, InIter first2, InIter last2,
                        OutIter result);
OutIter set_union(InIter first1, InIter last1, InIter first2, InIter last2,
                  OutIter result);
OutIter set_symmetric_difference(InIter first1, InIter last1, InIter first2,
                                InIter last2, OutIter result);
```

Using Sorting Algorithms Part 1

Program 19.27 Using sorting algorithms

```
1  /*****
2   * A program that sorts a vector using sort algorithm      *
3   *****/
4  #include <vector>
5  #include <algorithm>
6  #include <iostream>
7  using namespace std;
8
9  // Definition of print function
10 void print(int value)
11 {
12     cout << value << " ";
13 }
14
15 int main()
16 {
17     // Instantiation of a vector object
18     vector <int> vec ;
19     // Pushing six elements into the vector and print them
20     vec.push_back(17);
```

Using Sorting Algorithms Part 2

Program 19.27 Using sorting algorithms

```
21     vec.push_back(10);
22     vec.push_back(13);
23     vec.push_back(18);
24     vec.push_back(15);
25     vec.push_back(11);
26     cout << "Original vector" << endl;
27     for_each(vec.begin(), vec.end(), print);
28     cout << endl << endl;
29     // Sorting the vector in ascending order and print it
30     cout << "Vector after sorting in ascending order" << endl;
31     sort(vec.begin(), vec.end());
32     for_each(vec.begin(), vec.end(), print);
33     cout << endl << endl;
34     // Sorting the vector in descending order and print it
35     cout << "Vector after sorting in descending order" << endl;
36     sort(vec.begin(), vec.end(), greater<int>());
37     for_each(vec.begin(), vec.end(), print);
38     cout << endl << endl;
39     return 0;
40 }
```

Using Sorting Algorithms Part 3

Program 19.27 *Using sorting algorithms*

Run:

Original vector

17 10 13 18 15 11

Vector after sorting in ascending order

10 11 13 15 17 18

Vector after sorting in descending order

18 17 15 13 11 10

Using Binary Search Algorithm Part 1

Program 19.28 Using a binary search algorithm

```
1  /*****
2  * A program that uses binary search on a vector          *
3  *****/
4  #include <vector>
5  #include <algorithm>
6  #include <iostream>
7  using namespace std;
8
9  int main()
10 {
11     // Instantiation of a vector object
12     vector <int> vec ;
13     // Adding six elements to the vector
14     vec.push_back(17) ;
15     vec.push_back(10) ;
16     vec.push_back(13) ;
17     vec.push_back(18) ;
18     vec.push_back(15) ;
19     vec.push_back(11) ;
20     // Sorting the vector
```

Using Binary Search Algorithm Part 2

Program 19.28 Using a binary search algorithm

```
21     sort(vec.begin(), vec.end());  
22     // Searching vector for two values  
23     cout << "Found 10 in vector? " << boolalpha;  
24     cout << binary_search(vec.begin(), vec.end(), 10) << endl;  
25     cout << "Found 19 in vector? " << boolalpha;  
26     cout << binary_search(vec.begin(), vec.end(), 19) << endl;  
27     return 0;  
28 }
```

Run:

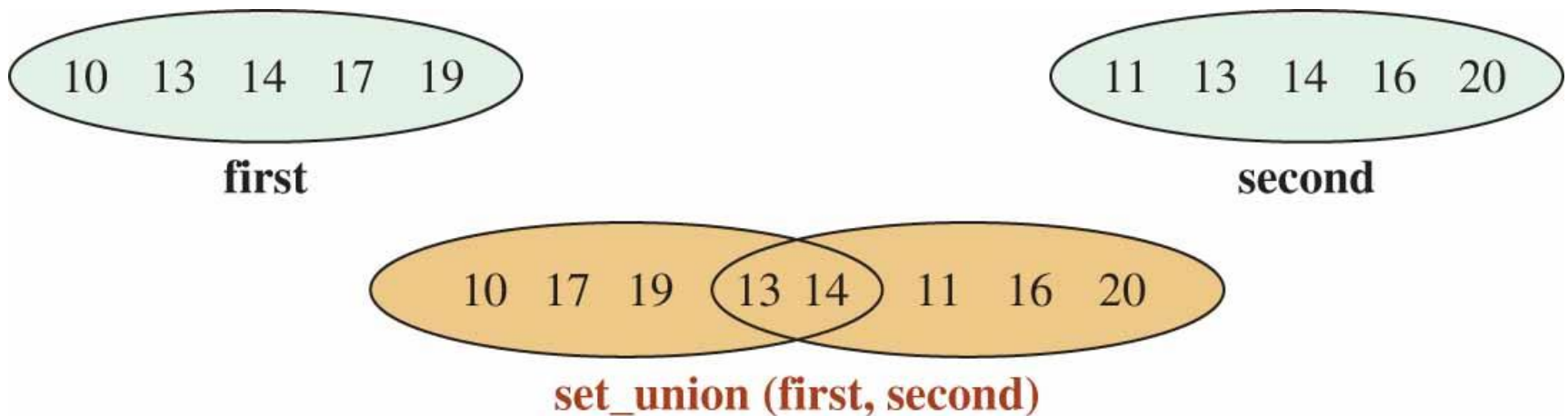
```
Found 10 in vector? true  
Found 19 in vector? false
```


Sorting and Related Algorithms Part 2

Figure 19.16 shows union operations applied to *first* and *second* sets.

The result is the colored area. The union is a set that includes all elements in first and second.

Figure 19.16 *Graphical representation of set union*



Program to Find the Union of Two Sets

Part 1

Program 19.29 Finding the union of two sets

```
1  /*****
2   * A program that finds the union of two sets          *
3   *****/
4  #include <set>
5  #include <vector>
6  #include <iostream>
7  #include <algorithm>
8  using namespace std;
9
10 // Print function
11 void print(int value)
12 {
13     cout << value << " ";
14 }
15
16 int main()
17 {
18     // Creation of the first set
19     set <int> first ;
20     first.insert(10);
```

Program to Find the Union of Two Sets

Part 2₍₁₎

Program 19.29 Finding the union of two sets

```
21     first.insert(19);
22     first.insert(14);
23     first.insert(17);
24     first.insert(13);
25     cout << "Elements of first set" << endl;
26     for_each(first.begin(), first.end(), print);
27     cout << endl << endl;
28     // Creation of the second set
29     set <int> second;
30     second.insert(16);
31     second.insert(14);
32     second.insert(13);
33     second.insert(11);
34     second.insert(20);
35     cout << "Elements of second set" << endl;
36     for_each(second.begin(), second.end(), print);
37     cout << endl << endl;
38     // Finding the union of two sets and storing in a vector
39     vector <int> temp(10);
40     vector <int> :: iterator iter;
```

Program to Find the Union of Two Sets

Part 2₍₂₎

Program 19.29 Finding the union of two sets

```
41     vector <int> :: iterator endIter;
42     endIter = set_union(first.begin(), first.end(), second.begin(),
43                        second.end(), temp.begin());
44     // Copying elements from the vector to the result set
45     set <int> result;
46     for (iter = temp.begin(); iter != endIter; iter++)
47     {
48         result.insert(*iter);
49     }
50     cout << "Elements of result set" << endl;
51     for_each(result.begin(), result.end(), print);
52     cout << endl << endl;
53     return 0;
54 }
```

Program to Find the Union of Two Sets

Part 3

Program 19.29 *Finding the union of two sets*

Run:

Elements of first set

10 13 14 17 19

Elements of second set

11 13 14 16 20

Elements of result set

10 11 13 14 16 17 19 20

Sorting and Related Algorithms Part 3

There is a point we need to mention about Program 19.29. The size of the target sequence needs to be large enough to accommodate the result.

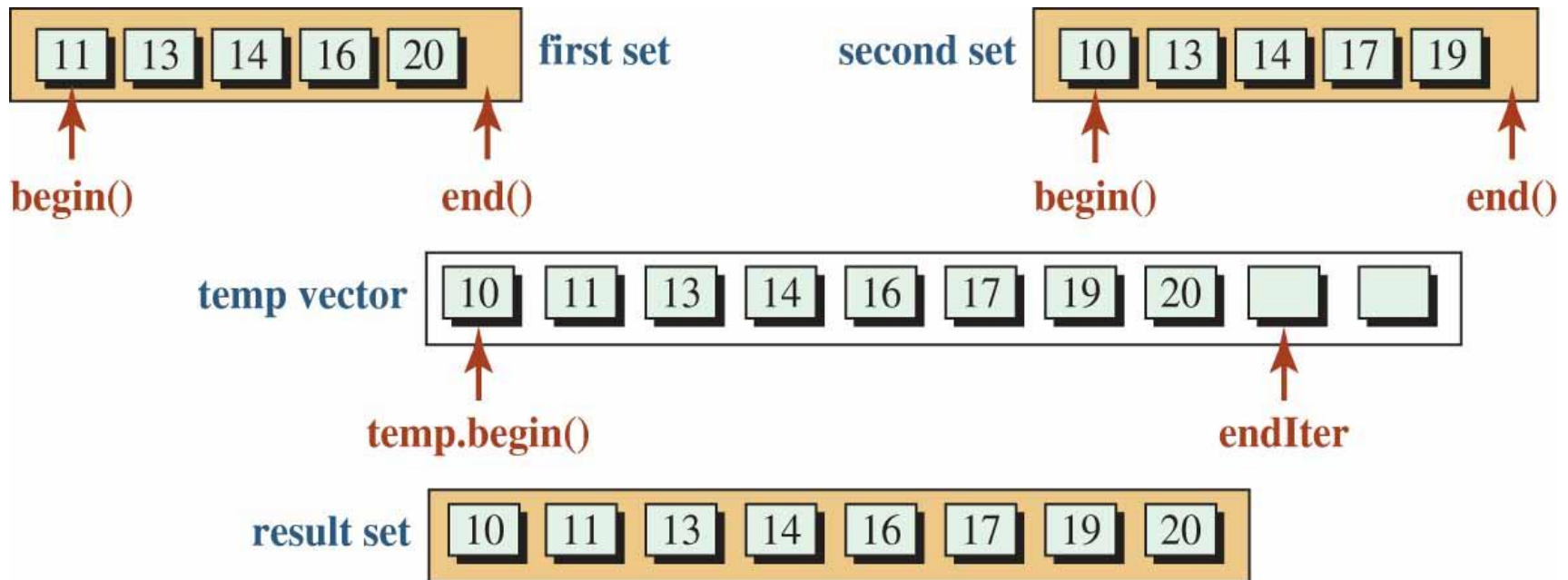
This is not possible with a set because any set when created is empty.

To solve the problem, we have created a vector to temporarily hold the result of an operation and then copied that part of the vector which holds the result to the resulting set.

Figure 19.17 shows the process we have used.

Sorting and Related Algorithms Part 4

Figure 19.17 *Process of using set_intersection operation*



Numeric Algorithms

There are a small number of algorithms, defined in the `<numeric>` header file, that perform simple arithmetic operations on the elements of a container or containers.

Note that these algorithms do not have to be applied on arithmetic types (such as *int* or *double*); as long as the corresponding operation is defined for a type, these algorithms can be applied to them.

Table 19.9 shows one commonly used numeric algorithms.

Table 19.9 *Sorting and related algorithm*

```
T accumulate(InIter first, InIter last, T init);
```

The *accumulate* algorithm finds the sum of the value in a range [*first*, *last*) and adds the result to the *init* value.

Testing Accumulate Algorithm Part 1

Program 19.30 Testing the accumulate algorithm

```
1  /*****
2   * A program to test numeric algorithm accumulate.          *
3   *****/
4  #include <vector>
5  #include <numeric>
6  #include <iostream>
7  using namespace std;
8
9  // A print function
10 void print(int value)
11 {
12     cout << value << " ";
13 }
14
15 int main()
16 {
17     // Instantiate and print a vector
18     vector <int> vec ;
19     vec.push_back(17);
20     vec.push_back(10);
```

Testing Accumulate Algorithm Part 2

Program 19.30 Testing the accumulate algorithm

```
21     vec.push_back(13);
22     vec.push_back(13);
23     vec.push_back(18);
24     vec.push_back(15);
25     vec.push_back(17);
26     for_each(vec.begin(), vec.end(), print);
27     cout << endl;
28     // Calculate the sum and print it
29     int sum = accumulate(vec.begin(), vec.end(), 0);
30     cout << "Sum of elements: " << sum;
31     return 0;
32 }
```

Run:

```
vector: 17  10  13  13  18  15  17
Sum of elements: 103
```